Planteamiento: soluciones

Transcribir textos, el bosque y dos colores

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Transcribir textos: bisección sobre la respuesta

2 El bosque: recorrido en amplitud sobre una rejilla

3 Dos colores: recorrido en profundidad modificado

4 Antes de programar

Contenido

1 Transcribir textos: bisección sobre la respuesta
- El modelo
- La traza
- El código
- Los bordes
- Lo que cuesta

El modelo: candidato y función



Qué se busca

El candidato es el tope t. La función es `alcanza`(t): ¿`empleados_necesarios`$(t) \leq k$? Es falsa hasta un borde y cierta de ahí en adelante. El borde es la respuesta, y la bisección lo encuentra.

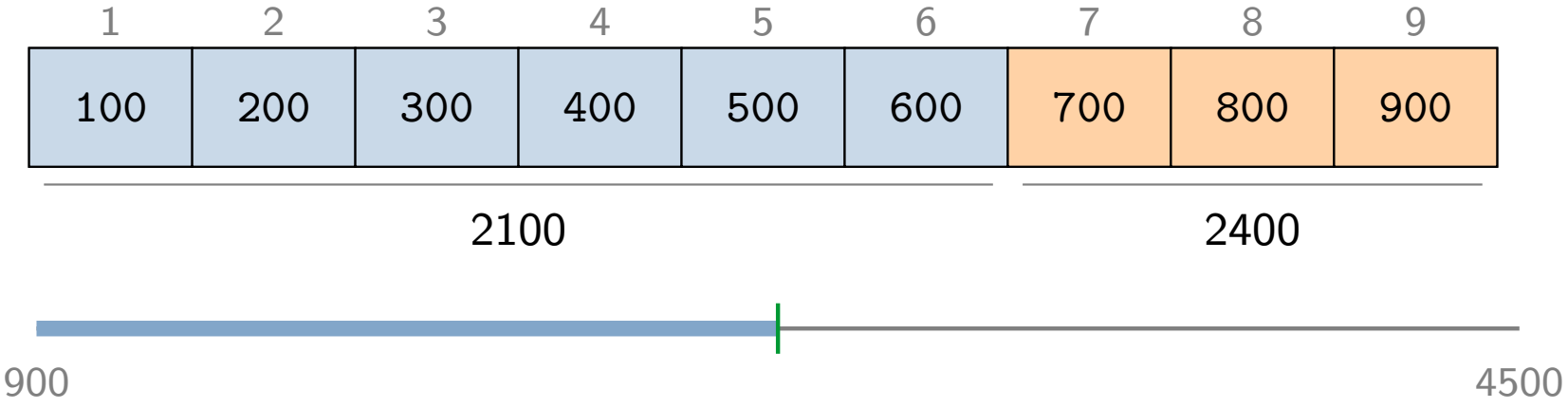
El modelo: los límites



Dónde arranca la búsqueda

Por abajo, $\texttt{lo} = \max p_i = 900$: el texto más largo va entero con alguien. Por arriba, $\texttt{hi} = \sum p_i = 4500$: un solo empleado lo hace todo. La respuesta siempre queda en ese intervalo.

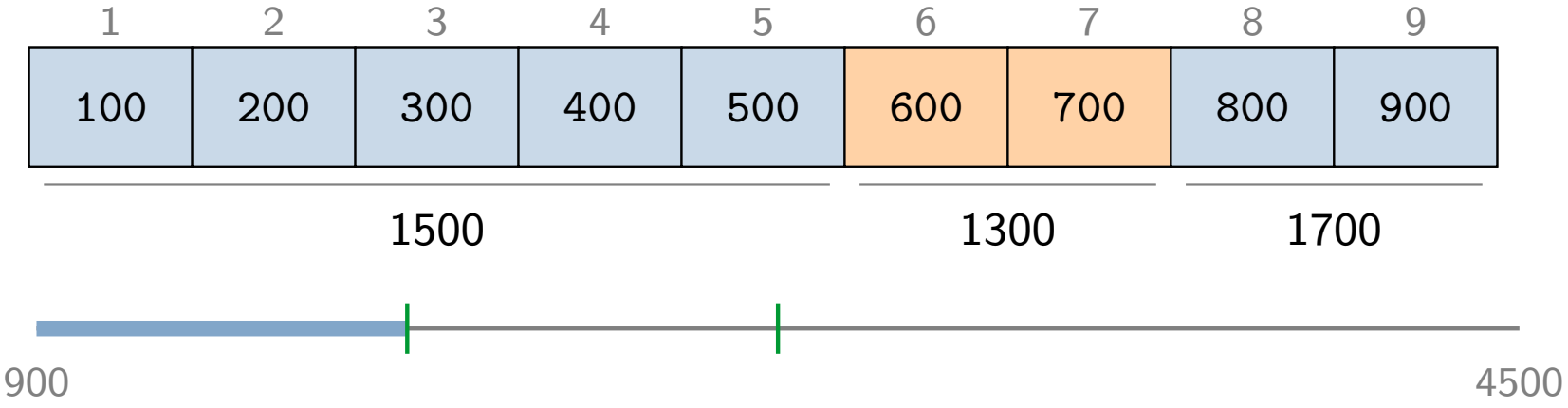
La traza: el tope y los cortes



Vuelta 1: se prueba `medio` = 2700

Antes, `lo` = 900 y `hi` = 4500. Hacen falta 2 y hay 3: alcanza, y `hi` baja a 2700.

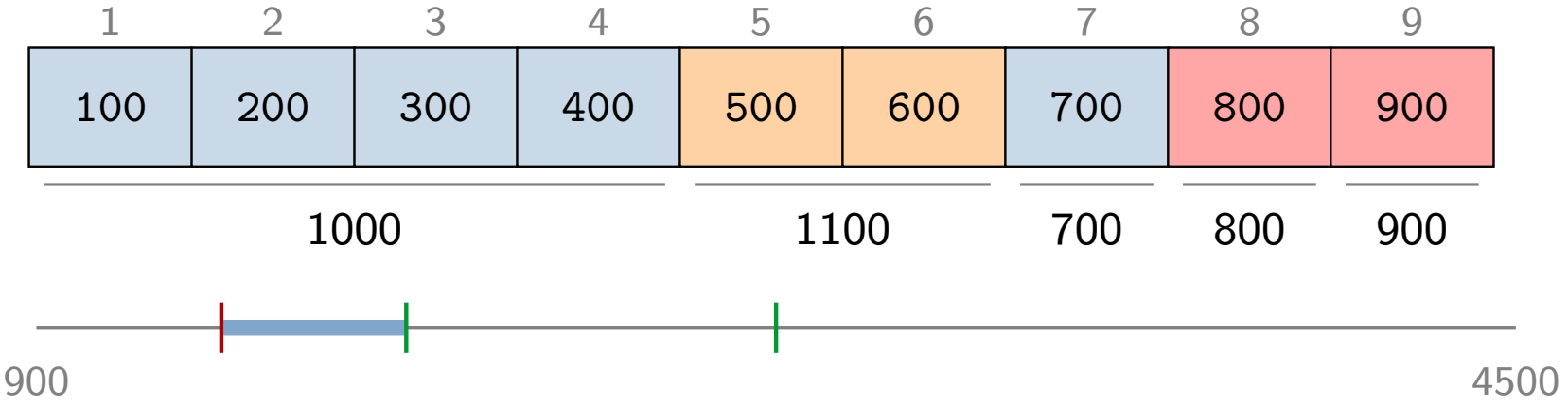
La traza: el tope y los cortes



Vuelta 2: se prueba `medio` = 1800

Antes, `lo` = 900 y `hi` = 2700. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1800.

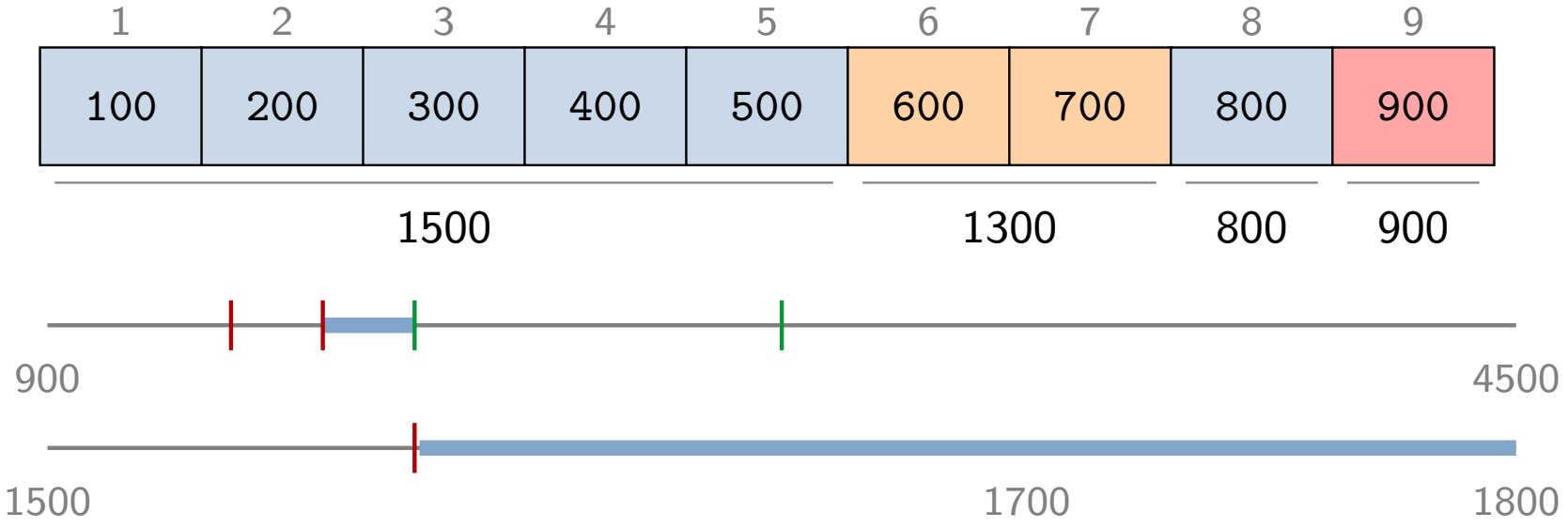
La traza: el tope y los cortes



Vuelta 3: se prueba `medio` = 1350

Antes, `lo` = 900 y `hi` = 1800. Hacen falta 5 y hay 3: no alcanza, y `lo` sube a 1351.

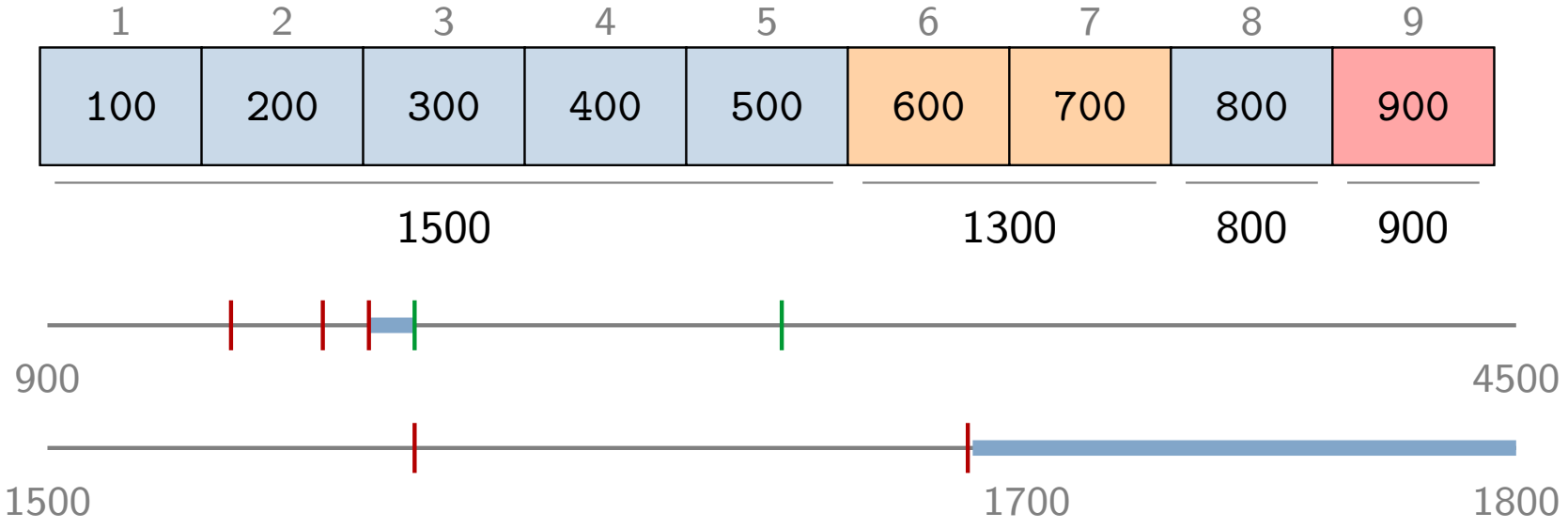
La traza: el tope y los cortes



Vuelta 4: se prueba `medio` $= 1575$

Antes, `lo` $= 1351$ y `hi` $= 1800$. Hacen falta 4 y hay 3: no alcanza, y `lo` sube a 1576.

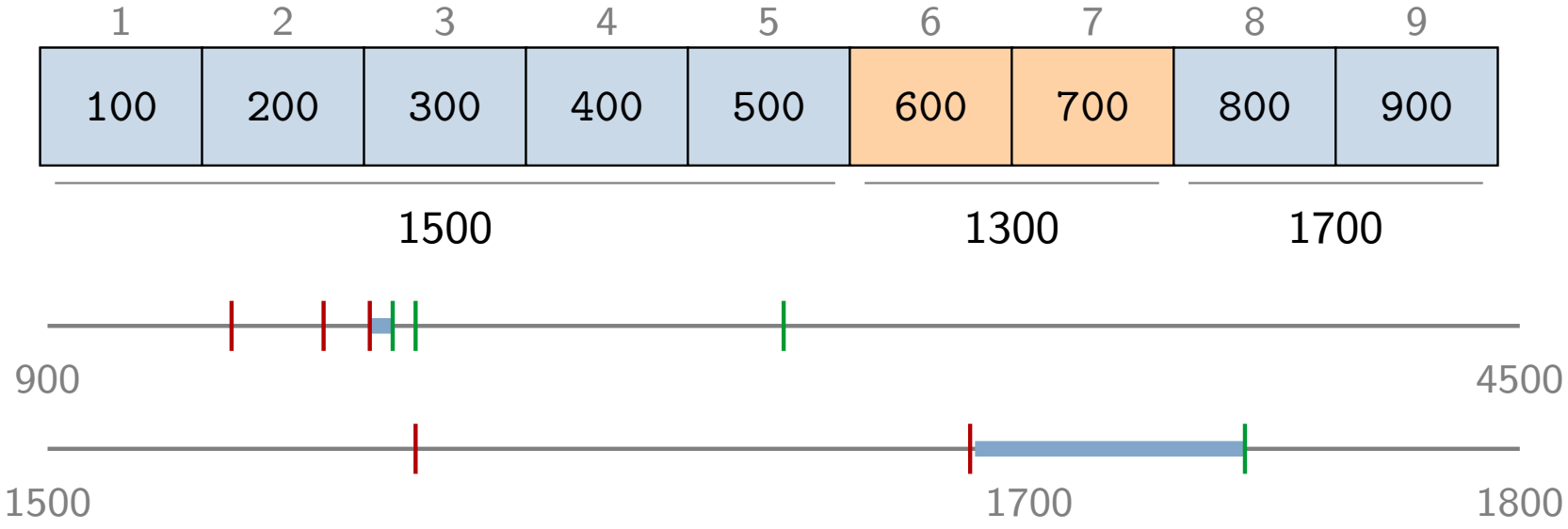
La traza: el tope y los cortes



Vuelta 5: se prueba `medio` $= 1688$

Antes, `lo` $= 1576$ y `hi` $= 1800$. Hacen falta 4 y hay 3: no alcanza, y `lo` sube a 1689.

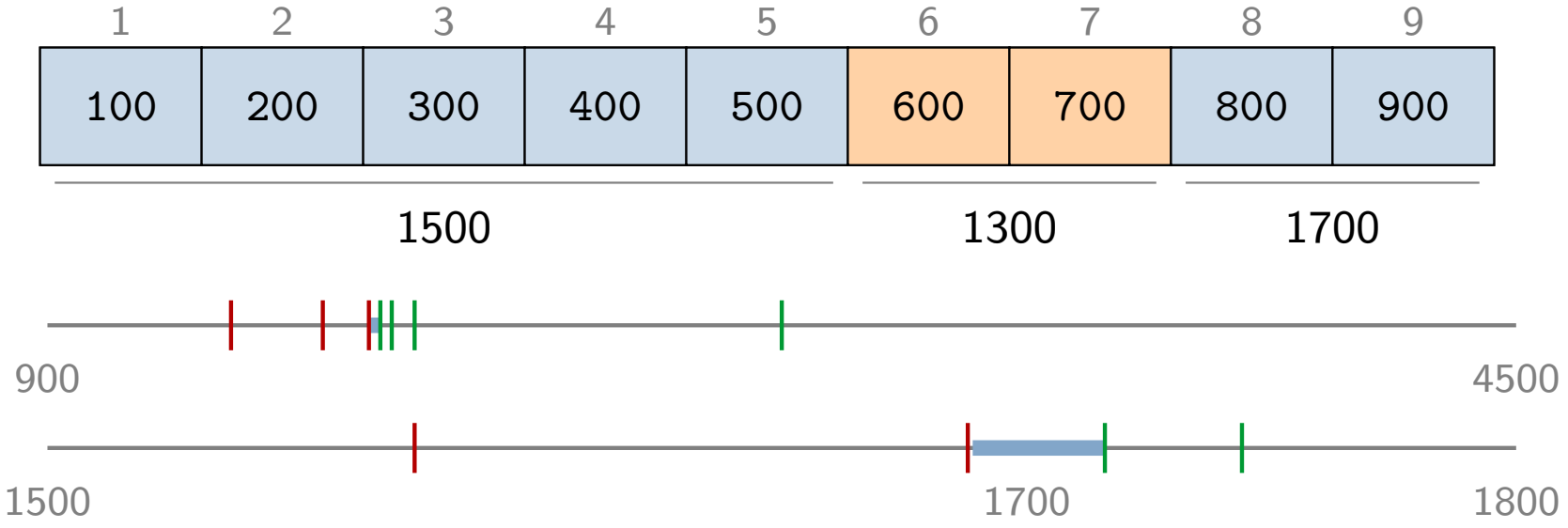
La traza: el tope y los cortes



Vuelta 6: se prueba `medio` $= 1744$

Antes, `lo` $= 1689$ y `hi` $= 1800$. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1744.

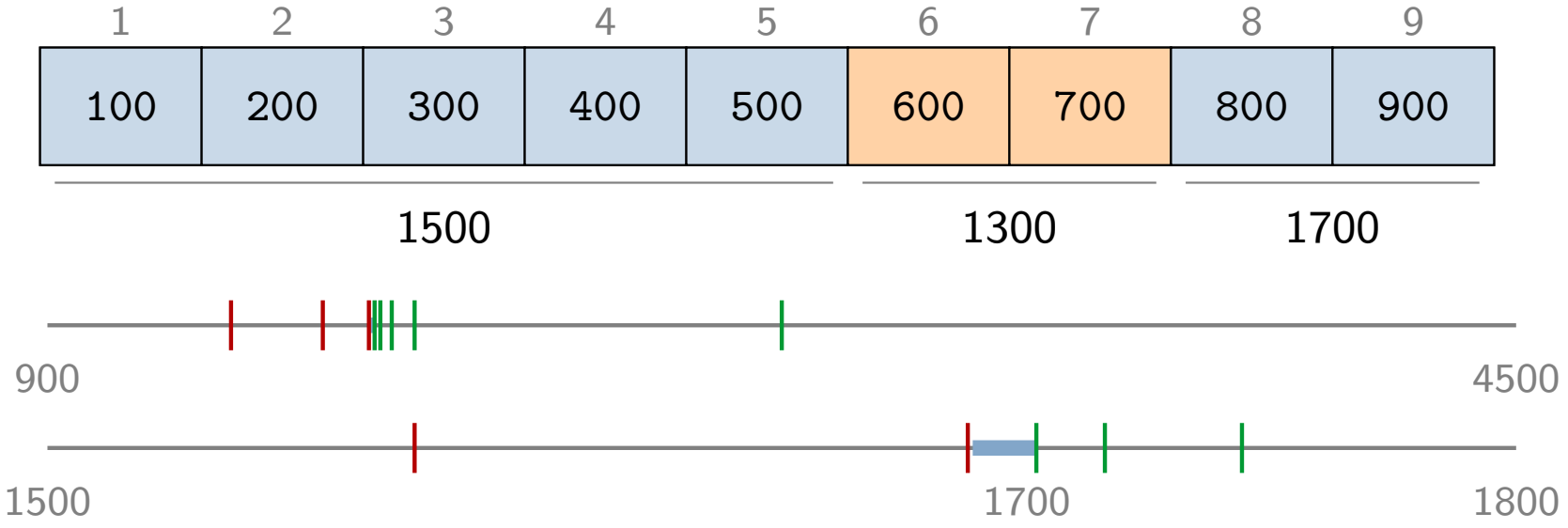
La traza: el tope y los cortes



Vuelta 7: se prueba `medio` $= 1716$

Antes, `lo` $= 1689$ y `hi` $= 1744$. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1716.

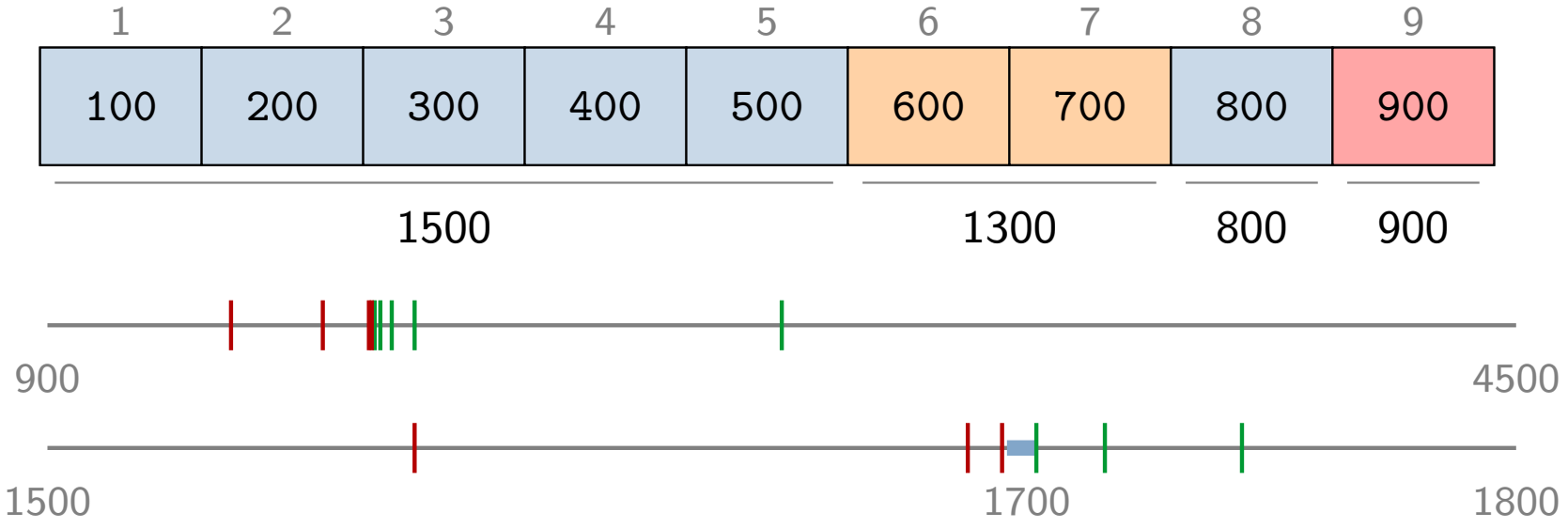
La traza: el tope y los cortes



Vuelta 8: se prueba `medio` $= 1702$

Antes, `lo` $= 1689$ y `hi` $= 1716$. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1702.

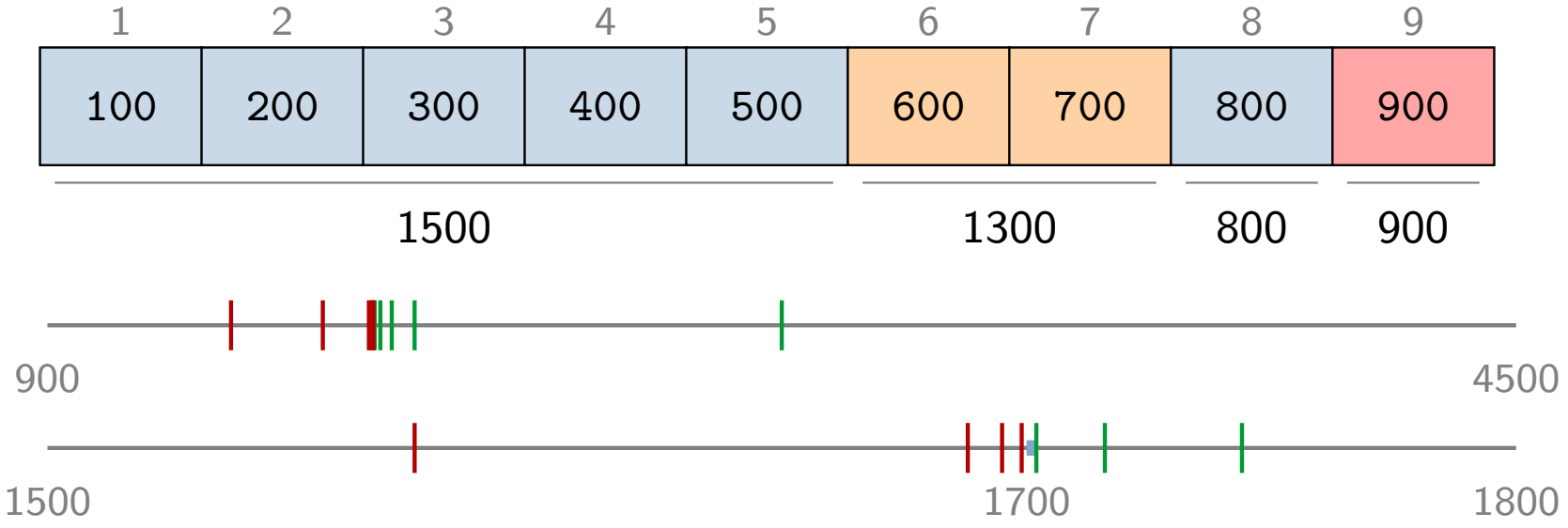
La traza: el tope y los cortes



Vuelta 9: se prueba `medio` $= 1695$

Antes, `lo` $= 1689$ y `hi` $= 1702$. Hacen falta 4 y hay 3: no alcanza, y `lo` sube a 1696.

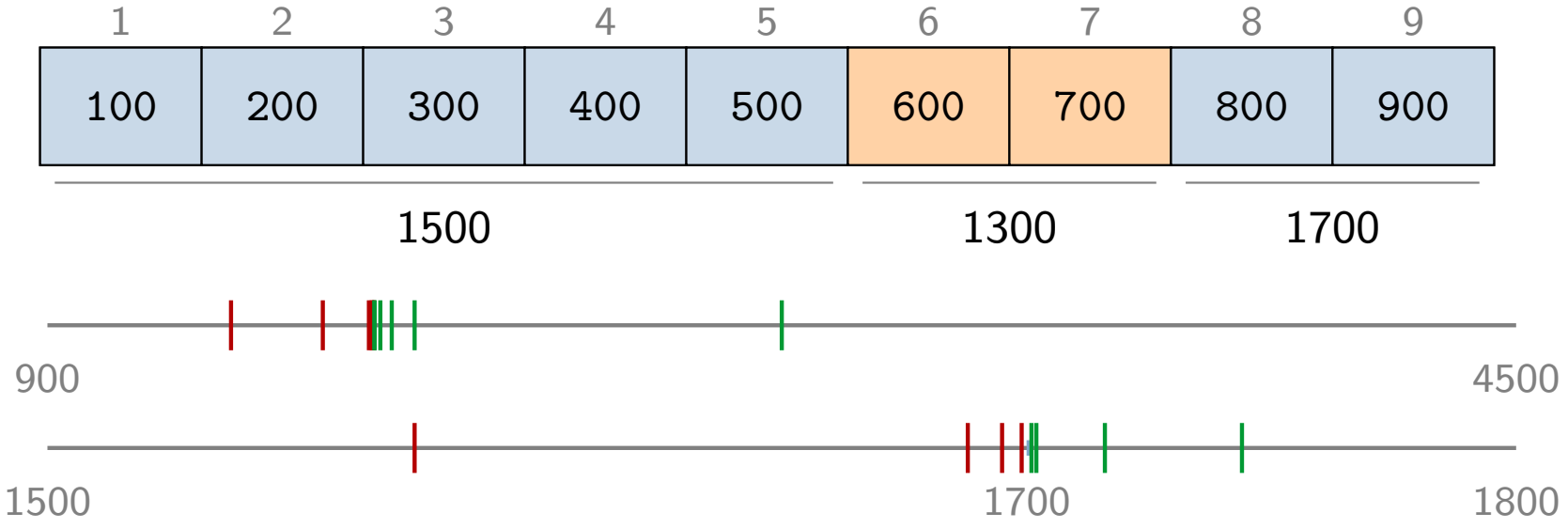
La traza: el tope y los cortes



Vuelta 10: se prueba `medio` $= 1699$

Antes, `lo` $= 1696$ y `hi` $= 1702$. Hacen falta 4 y hay 3: no alcanza, y `lo` sube a 1700.

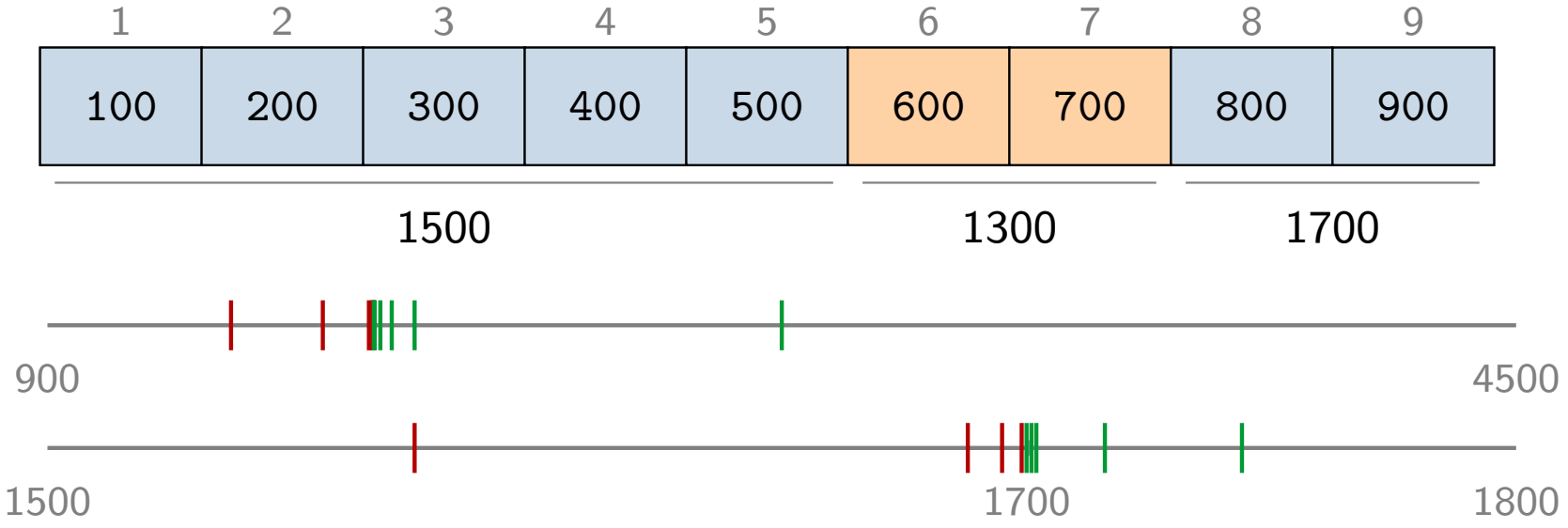
La traza: el tope y los cortes



Vuelta 11: se prueba `medio` $= 1701$

Antes, `lo` $= 1700$ y `hi` $= 1702$. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1701.

La traza: el tope y los cortes



Vuelta 12: se prueba `medio` $= 1700$

Antes, `lo` $= 1700$ y `hi` $= 1701$. Hacen falta 3 y hay 3: alcanza, y `hi` baja a 1700.

La traza: el tope y los cortes

Vuelta	`lo`	`hi`	`medio`	Empleados	¿Alcanza?
1	900	4500	2700	2	sí
2	900	2700	1800	3	sí
3	900	1800	1350	5	no
4	1351	1800	1575	4	no
5	1576	1800	1688	4	no
6	1689	1800	1744	3	sí
7	1689	1744	1716	3	sí
8	1689	1716	1702	3	sí
9	1689	1702	1695	4	no
10	1696	1702	1699	4	no
11	1700	1702	1701	3	sí
12	1700	1701	1700	3	sí

El intervalo se cierra en 1700, la respuesta.

Antes del código: cómo se mueven los límites



Qué dice cada respuesta

Si `medio` alcanza, puede ser la respuesta: `hi = medio` lo deja dentro del intervalo. Si no alcanza, la respuesta es mayor: `lo = medio + 1` lo saca.

El código en Python: cuántos empleados

```
def empleados_necesarios(paginas, t):
    # Empleados necesarios si nadie pasa de t páginas.
    empleados = 1
    turno = 0
    for p in paginas:
        if turno + p <= t:
            turno = turno + p
        else:
            empleados = empleados + 1
            turno = p
    return empleados
```

El código en Python: la bisección

```
def transcribir(paginas, k):
    # El menor máximo de páginas por empleado.
    lo = max(paginas)
    hi = sum(paginas)
    while lo < hi:
        medio = (lo + hi) // 2
        if empleados_necesarios(paginas, medio) <= k:
            hi = medio
        else:
            lo = medio + 1
    return lo
```

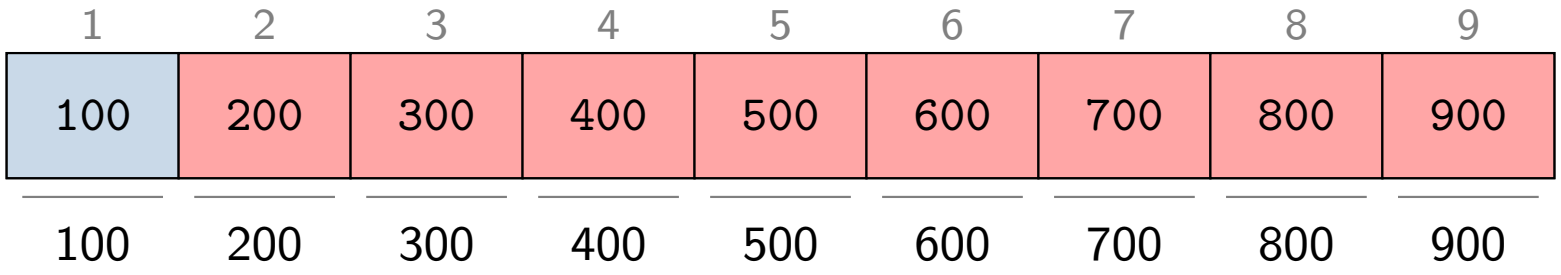
El código en C++: cuántos empleados

```cpp
int empleados_necesarios(const vector<long long> &paginas, long long t) {
  // Empleados necesarios si nadie pasa de t páginas.
  int empleados = 1;
  long long turno = 0;
  for (size_t i = 0; i < paginas.size(); i++) {
    if (turno + paginas[i] <= t) {
      turno = turno + paginas[i];
    } else {
      empleados = empleados + 1;
      turno = paginas[i];
    }
  }
  return empleados;
}
```

El código en C++: la bisección

```cpp
long long transcribir(const vector<long long> &paginas, int k) {
  // El menor máximo de páginas por empleado.
  long long lo = 0;
  long long hi = 0;
  for (size_t i = 0; i < paginas.size(); i++) {
    if (paginas[i] > lo) {
      lo = paginas[i];
    }
    hi = hi + paginas[i];
  }
  while (lo < hi) {
    long long medio = (lo + hi) / 2;
    if (empleados_necesarios(paginas, medio) <= k) {
      hi = medio;
    } else {
      lo = medio + 1;
    }
  }
  return lo;
}
```

Los bordes: arrancar con `lo` $= 0$



Con $k = 9$ responde 100

Con $t = 100$, cada texto siguiente no cabe en el turno y abre un empleado, sin mirar si cabe en t: nueve empleados, uno por texto. Cuenta $9 \leq 9$ y dice que alcanza, con textos de hasta 900 páginas. Por eso `lo` arranca en el texto más largo.

Los bordes: pocos y muchos empleados

Caso	Sale	Por qué
$k = 1$	4500	un empleado suma todo
$k = M$	900	cada texto con el suyo
$k > M$	900	sobran empleados

Lo que cuesta

De qué depende

Sea $S = \sum p_i - \max p_i$, el ancho del intervalo. La bisección da $\lceil \lg S \rceil$ vueltas, y cada vuelta recorre los M textos una vez:

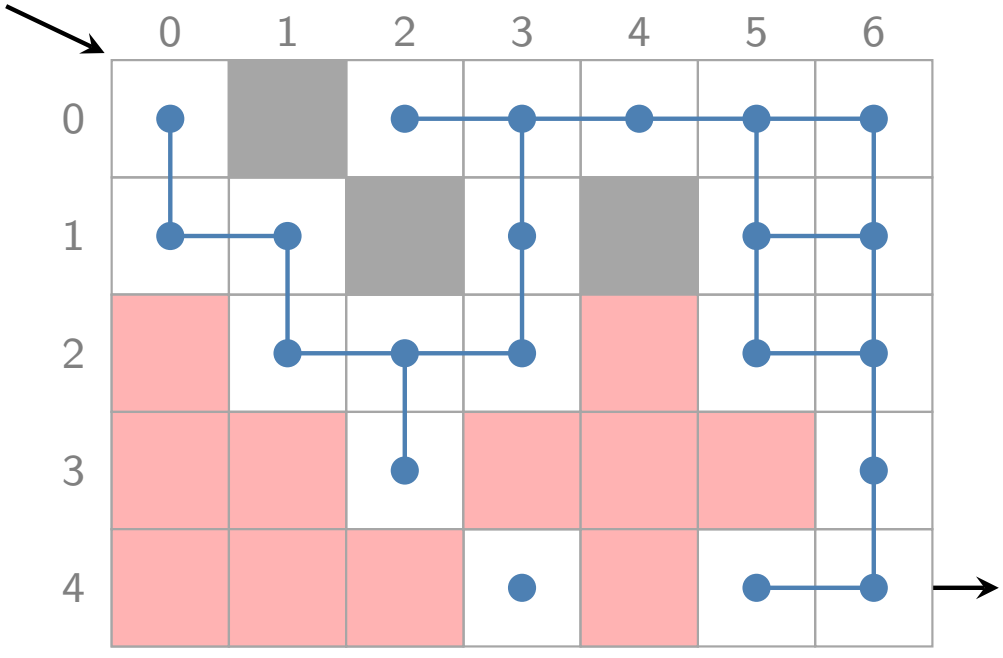
$$\Theta(M \lg S).$$

Aquí $S = 3600$ y fueron 12 vueltas sobre 9 textos. Para sacar los cortes basta una pasada más con t igual a la respuesta.

Contenido

2 El bosque: recorrido en amplitud sobre una rejilla
- El modelo
- La traza
- El código
- Los bordes
- Lo que cuesta

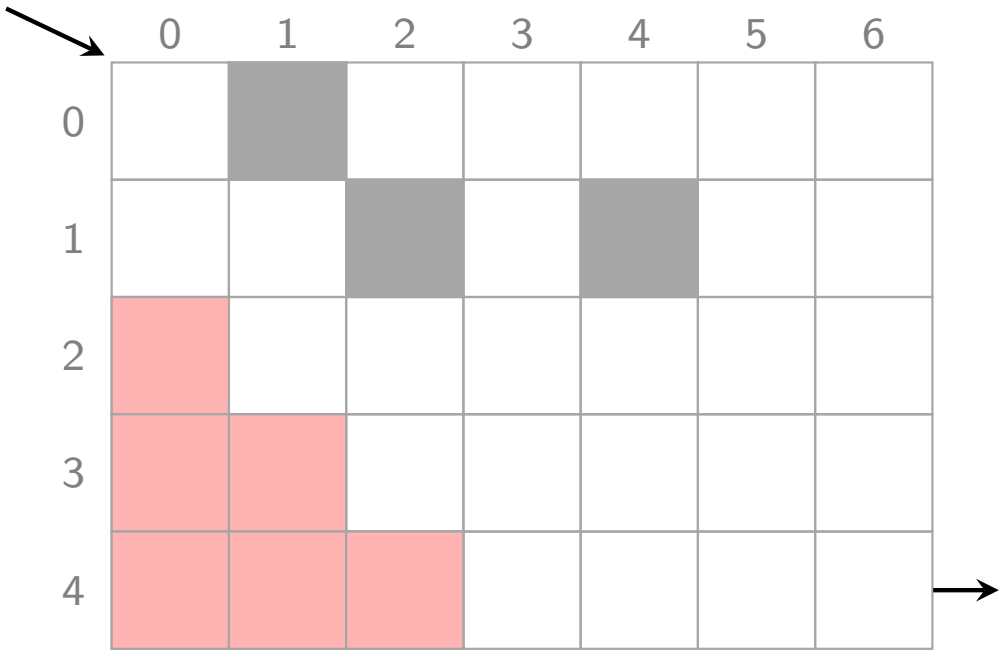
El modelo: nodos y aristas



Qué es cada cosa

Un nodo por casilla que se puede pisar. Una arista entre vecinas de arriba, abajo o a los lados. El recorrido en amplitud visita por capas desde la entrada: la capa d son las casillas a d pasos.

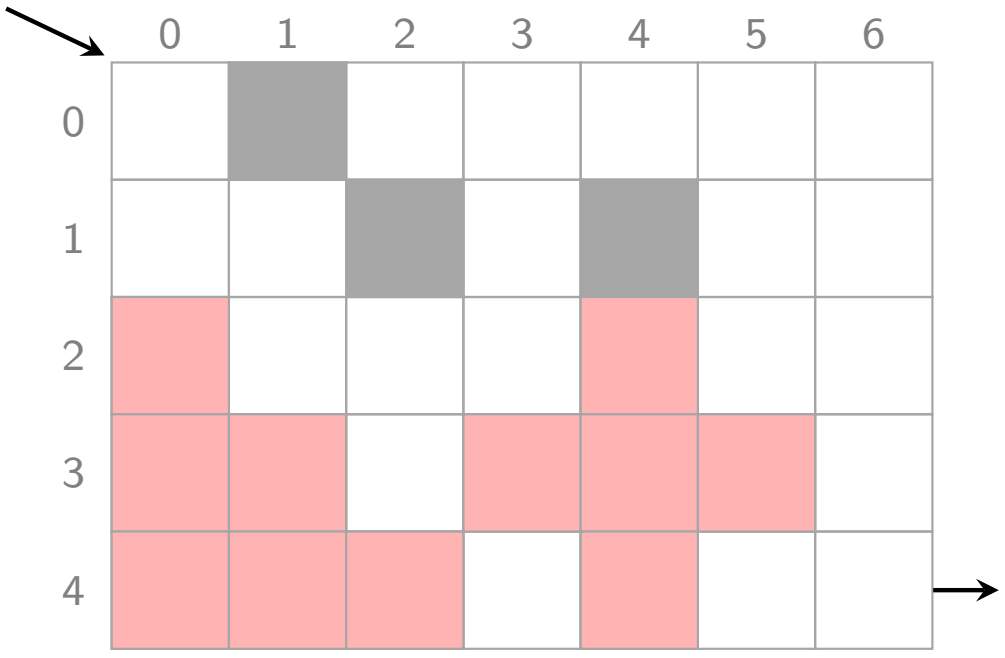
La traza: primero marcar



Tripleta $(4, 0, 2)$

Se pintan las casillas con $(r-4)^2 + (c-0)^2 \leq 4$:
`(2,0)`, `(3,0)`, `(3,1)`,
`(4,0)`, `(4,1)`, `(4,2)`.

La traza: primero marcar



Tripleta $(3, 4, 1)$

Se pintan las casillas con $(r-3)^2 + (c-4)^2 \leq 1$:
`(2,4)`, `(3,3)`, `(3,4)`, `(3,5)`, `(4,4)`.

La traza: la regla de la cola

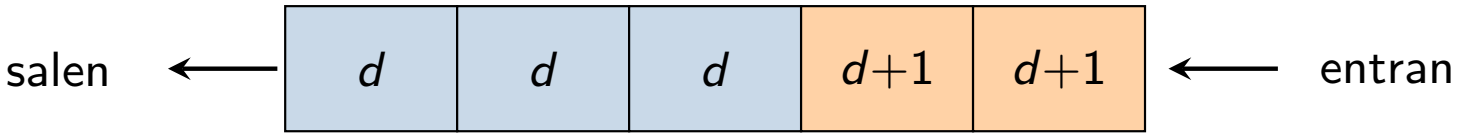
Una vuelta del recorrido

1. La entrada arranca en la cola, con número 0.
2. Sale la casilla del frente, con su número d.
3. Entran sus vecinas que se pueden pisar y aún no tienen número, en orden arriba, abajo, izquierda, derecha.
4. Cada una recibe $d + 1$, y se repite hasta vaciar la cola.



El número de cada casilla es su distancia a la entrada, en pasos.

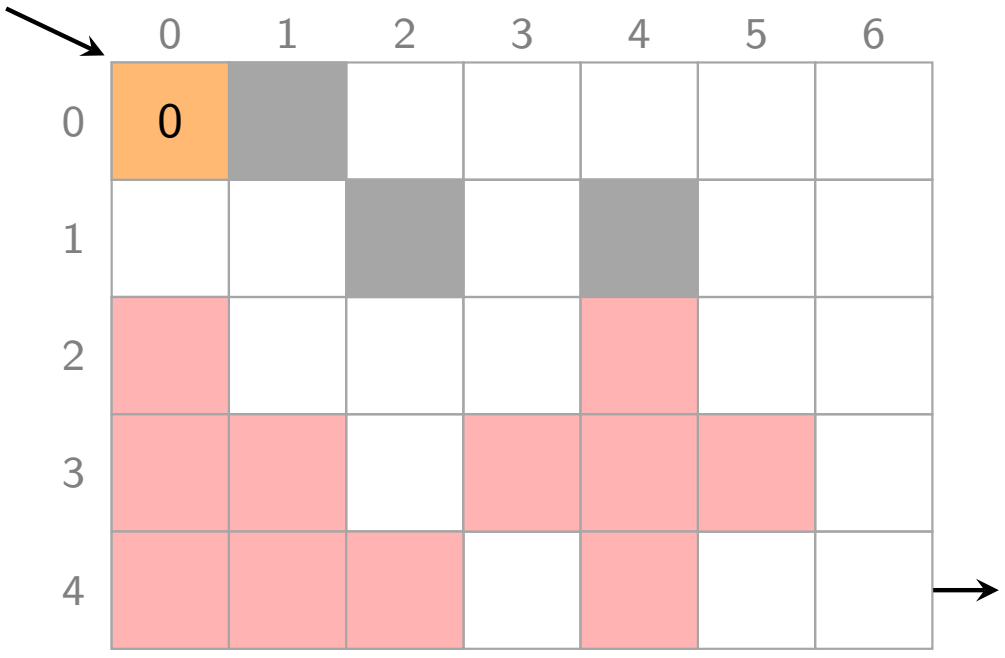
La traza: por qué la cola da la distancia menor



Salen en orden de distancia

Quien sale con d mete a sus vecinas con $d+1$, al final de la cola. Por eso todas las de d salen antes que cualquiera de $d+1$. La primera que toca a una casilla es entonces su vecina más cercana a la entrada, y ese primer toque es el camino más corto. Los toques posteriores no lo mejoran y se ignoran.

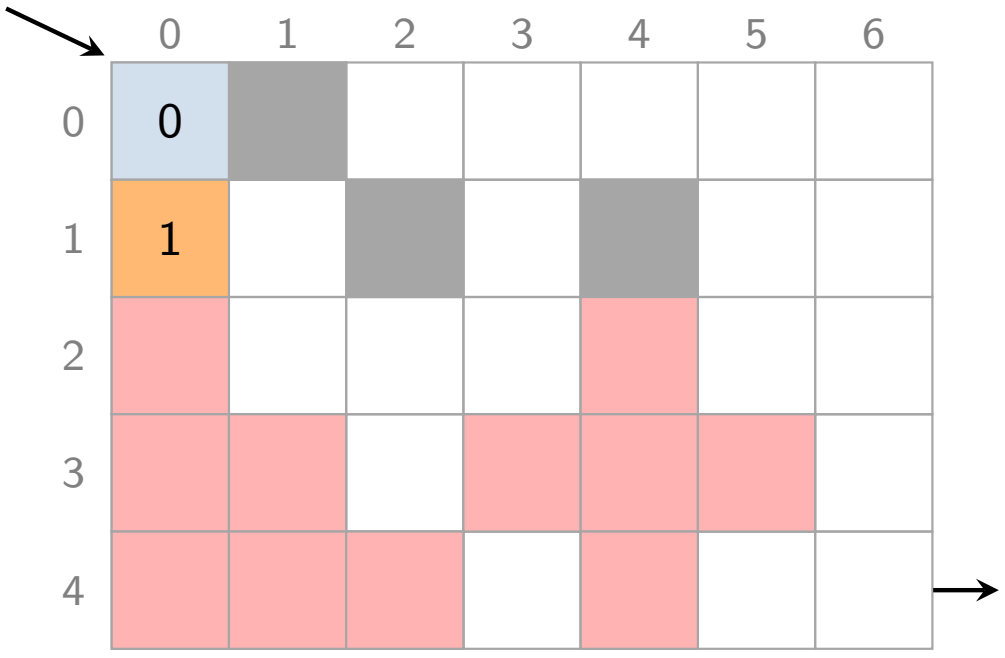
La traza: el frente de la cola



Sale	Entran
	(0,0)

Capa $d = 0$. Cola: (0,0).

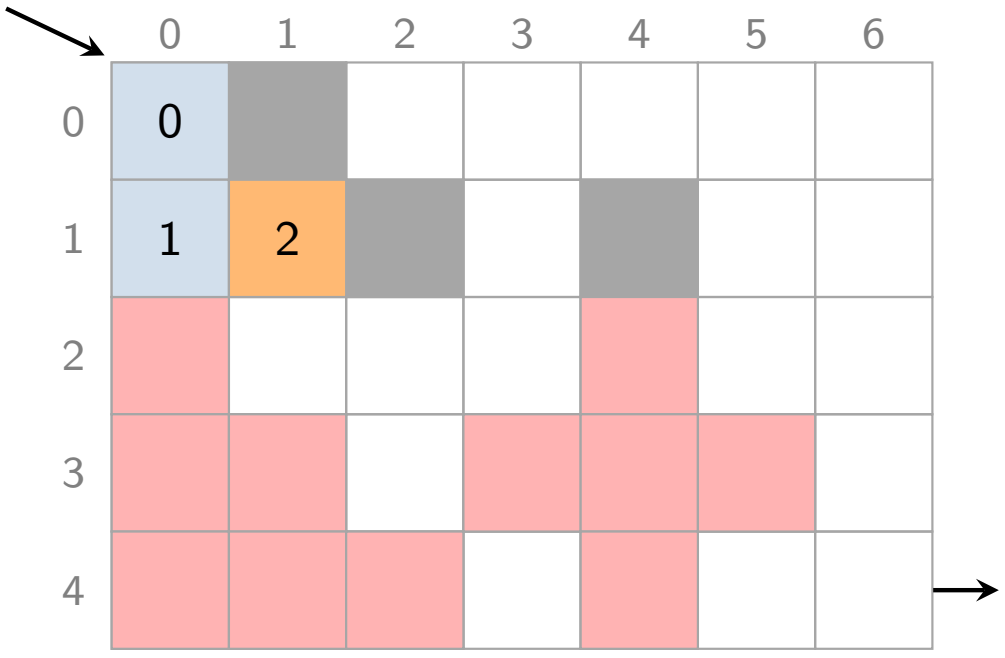
La traza: el frente de la cola



Sale	Entran
(0,0)	(1,0)

Capa $d = 1$. Cola: (1,0).

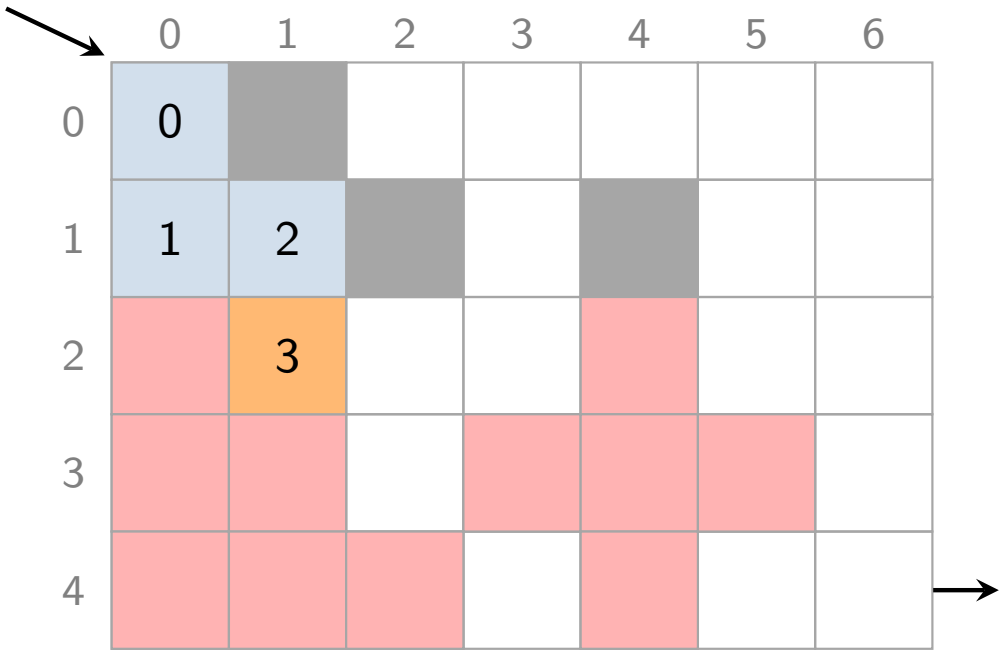
La traza: el frente de la cola



Sale	Entran
(1,0)	(1,1)

Capa $d = 2$. Cola: (1,1).

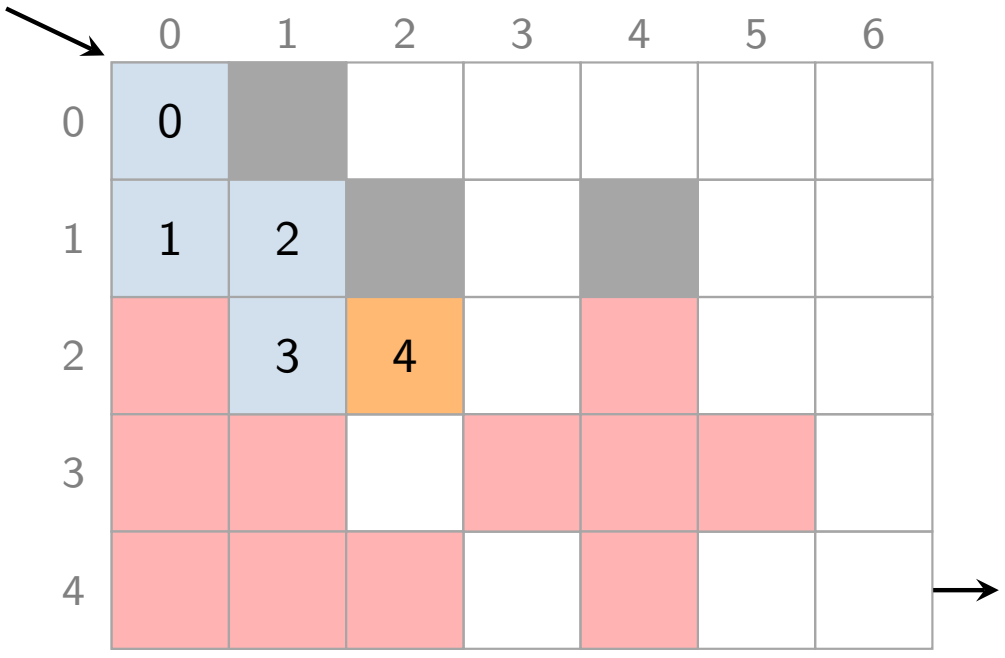
La traza: el frente de la cola



Sale	Entran
(1,1)	(2,1)

Capa $d = 3$. Cola: (2,1).

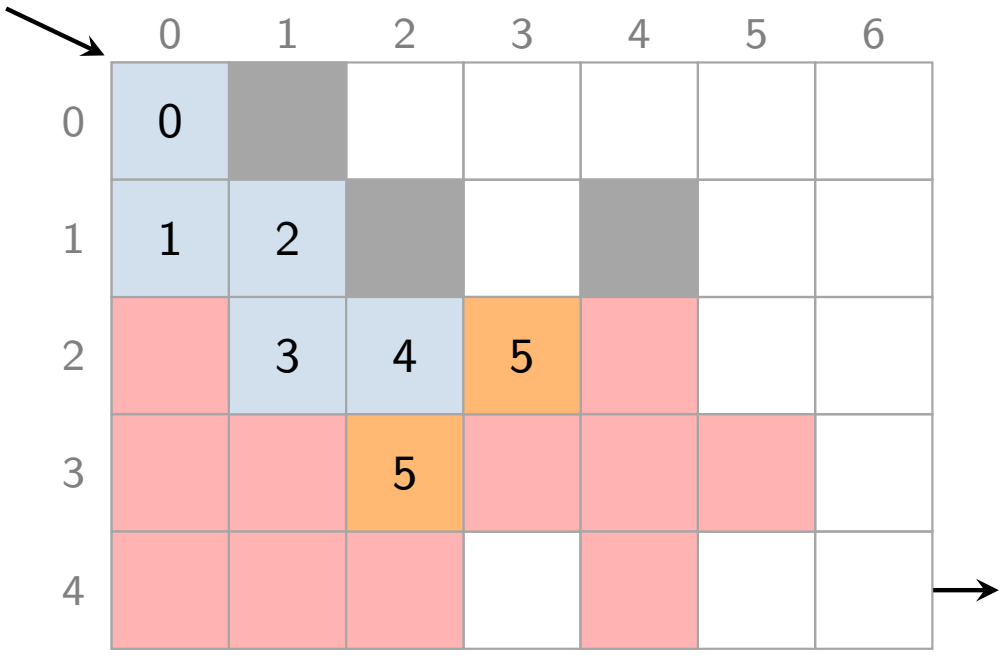
La traza: el frente de la cola



Sale	Entran
(2,1)	(2,2)

Capa $d = 4$. Cola: (2,2).

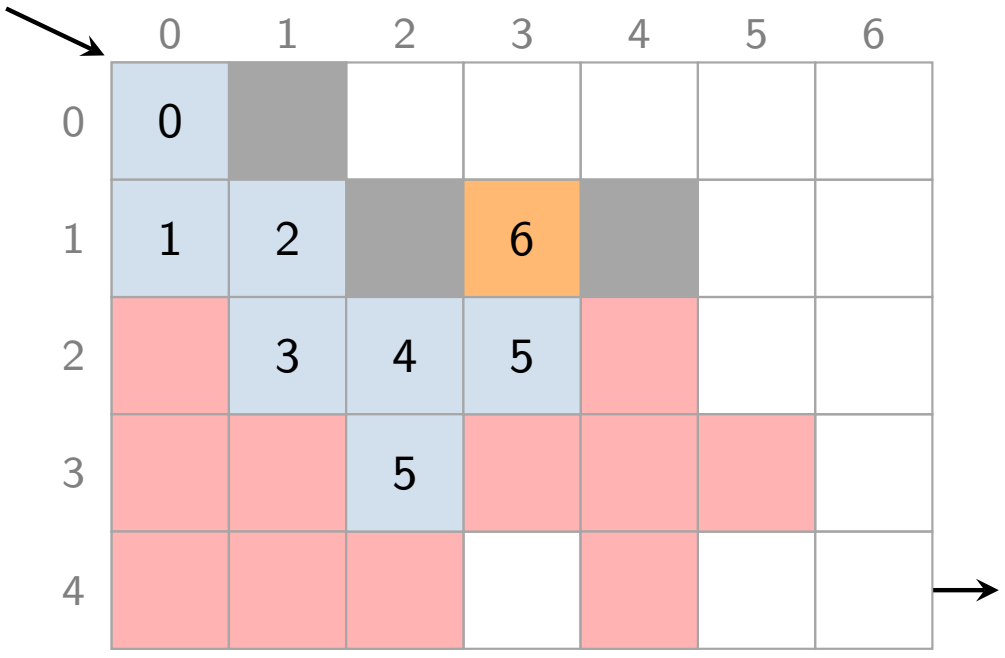
La traza: el frente de la cola



Sale	Entran
(2,2)	(3,2) (2,3)

Capa $d = 5$. Cola: (3,2) (2,3).

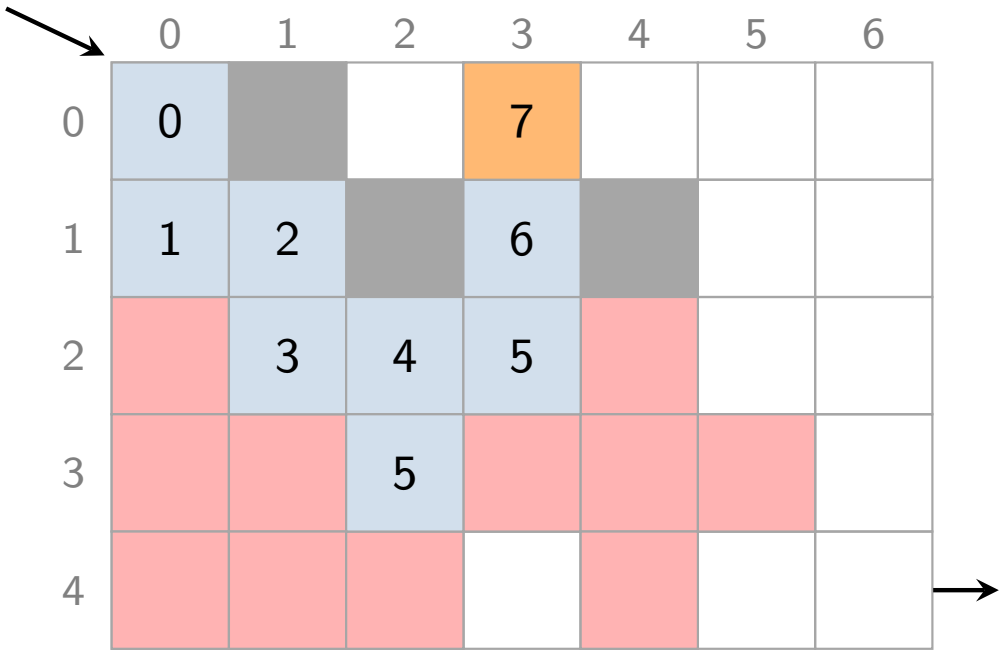
La traza: el frente de la cola



Sale	Entran
(3,2)	
(2,3)	(1,3)

Capa $d = 6$. Cola: (1,3).

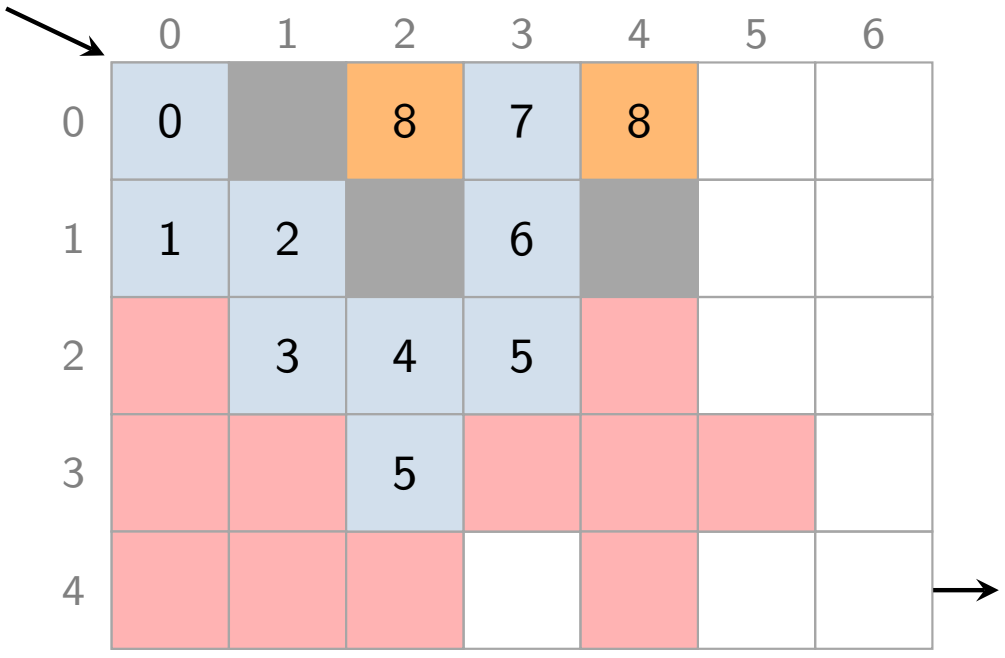
La traza: el frente de la cola



Sale	Entran
(1,3)	(0,3)

Capa $d = 7$. Cola: (0,3).

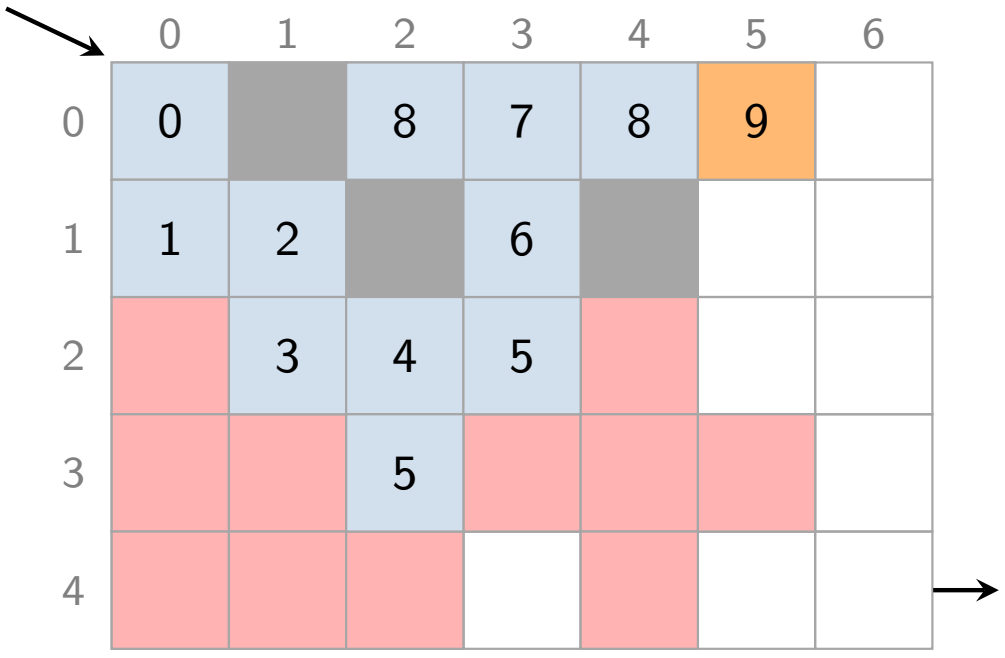
La traza: el frente de la cola



Sale	Entran
(0,3)	(0,2) (0,4)

Capa $d = 8$. Cola: (0,2) (0,4).

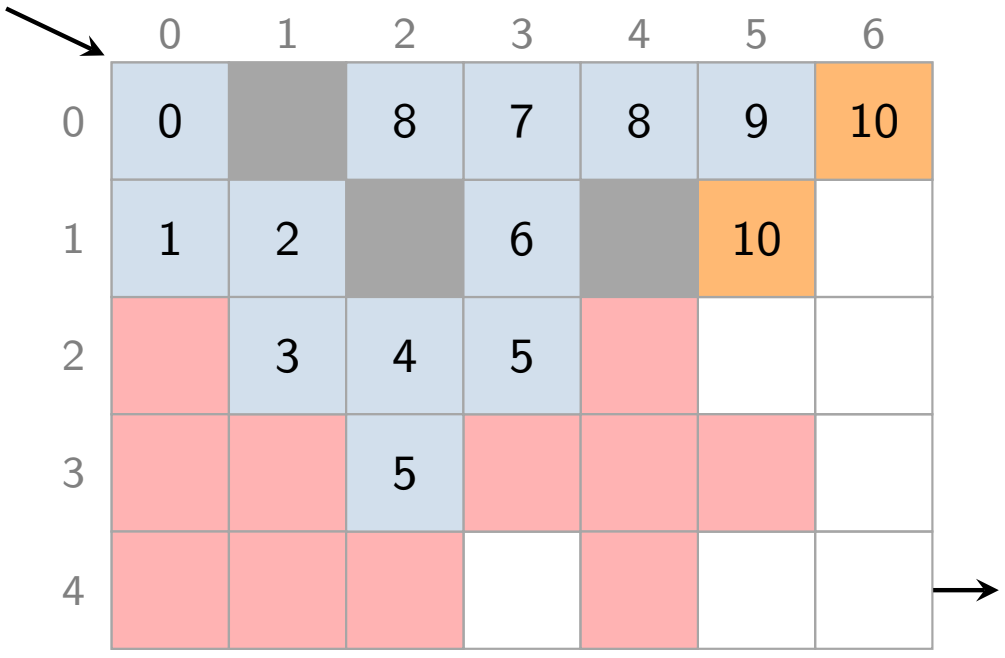
La traza: el frente de la cola



Sale	Entran
(0,2)	
(0,4)	(0,5)

Capa $d = 9$. Cola: (0,5).

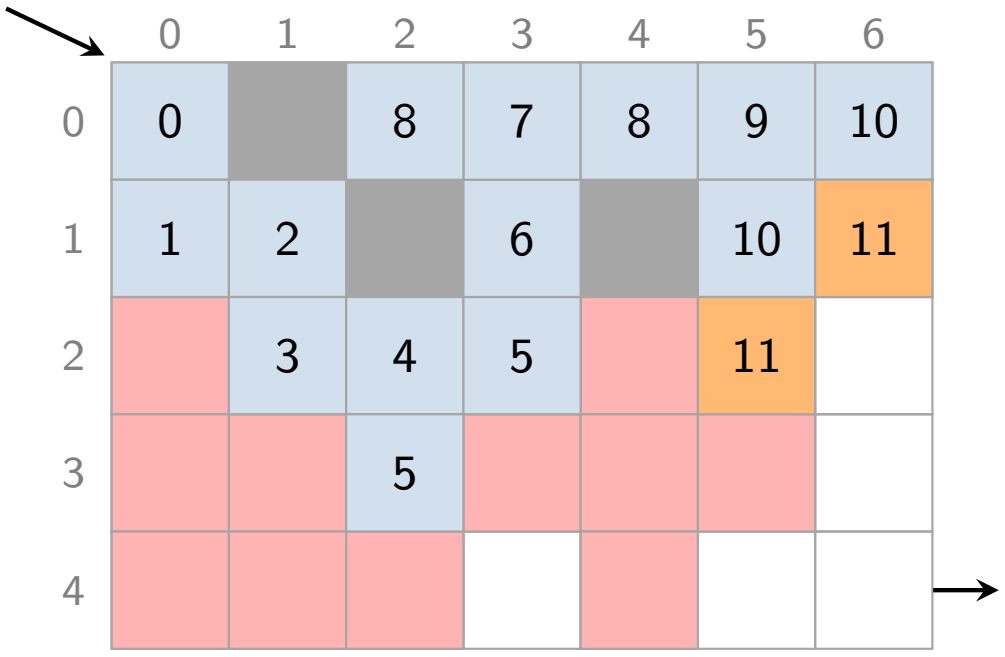
La traza: el frente de la cola



Sale	Entran
(0,5)	(1,5) (0,6)

Capa $d = 10$. Cola: (1,5) (0,6).

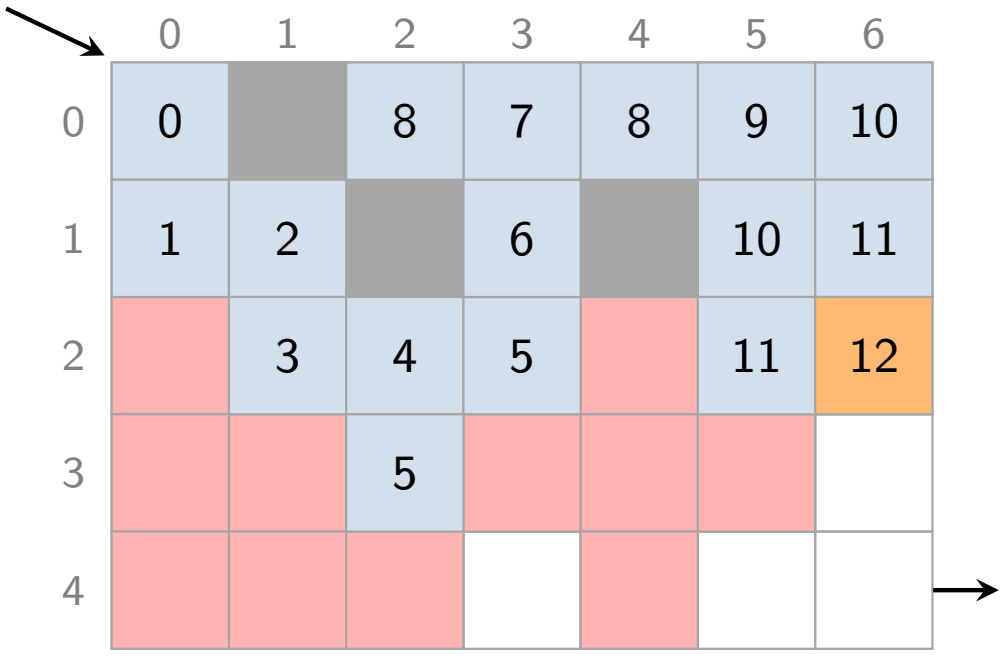
La traza: el frente de la cola



Sale	Entran
(1,5)	(2,5) (1,6)
(0,6)	

Capa $d = 11$. Cola: (2,5) (1,6).

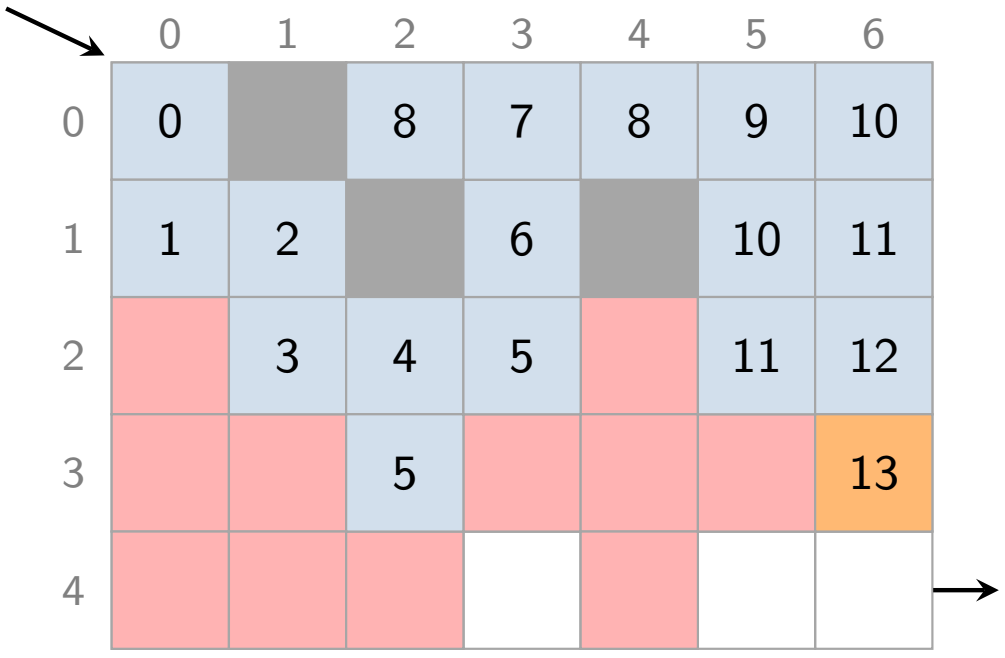
La traza: el frente de la cola



Sale	Entran
(2,5)	(2,6)
(1,6)	

Capa $d = 12$. Cola: (2,6).

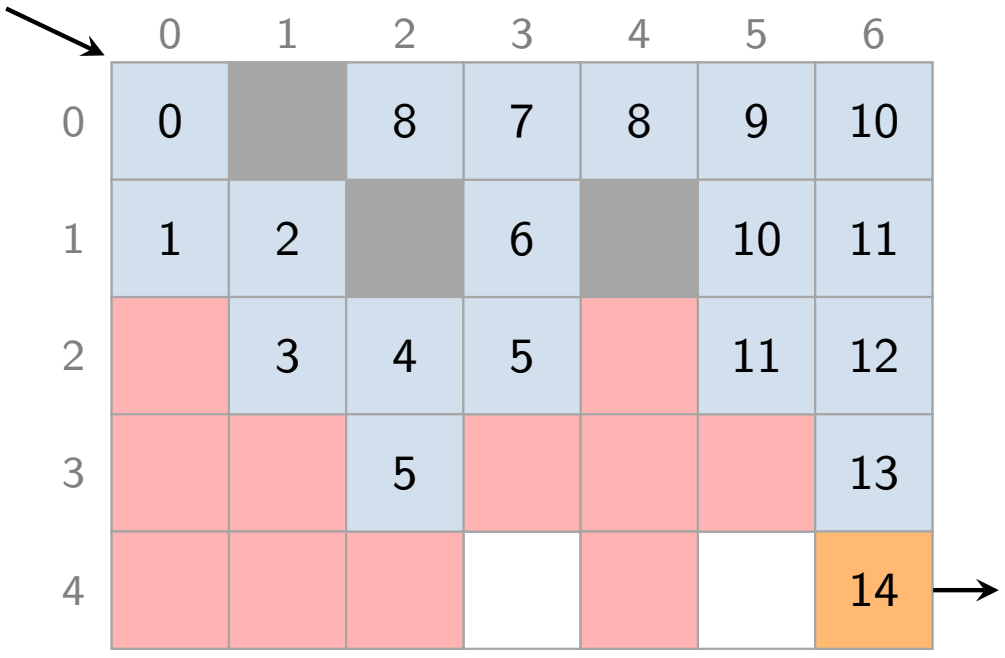
La traza: el frente de la cola



Sale	Entran
(2,6)	(3,6)

Capa $d = 13$. Cola: (3,6).

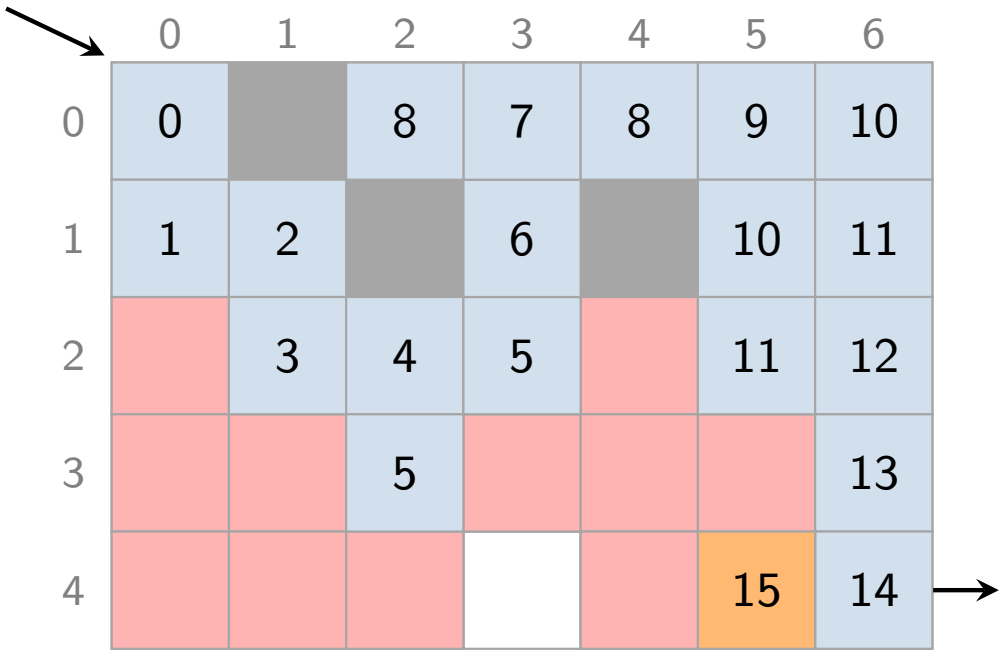
La traza: el frente de la cola



Sale	Entran
(3,6)	(4,6)

Capa $d = 14$. La salida entra con $d = 14$: son 14 pasos. El ciclo sigue hasta vaciar la cola.

La traza: el frente de la cola



Sale	Entran
(4,6)	(4,5)

Capa $d = 15$. Cola: (4,5). Al salir de ahí no entra nadie y el ciclo termina.

El código en Python: marcar

```python
def marcar(tablero, tripletas):
    # Matriz con las casillas por las que no se puede pasar.
    R = len(tablero)
    C = len(tablero[0])
    prohibida = []
    for r in range(R):
        fila = []
        for c in range(C):
            fila.append(tablero[r][c] == "#")
        prohibida.append(fila)
    for (ri, ci, radio) in tripletas:
        for r in range(max(0, ri - radio), min(R, ri + radio + 1)):
            for c in range(max(0, ci - radio), min(C, ci + radio + 1)):
                if (r - ri) * (r - ri) + (c - ci) * (c - ci) <= radio * radio:
                    prohibida[r][c] = True
    return prohibida
```

El código en Python: entrar a la cola

```python
def entrar(prohibida, dist, cola, r, c, d):
    # Mete (r, c) a la cola si existe, se puede pisar y es nueva.
    if 0 <= r < len(dist) and 0 <= c < len(dist[0]):
        if not prohibida[r][c] and dist[r][c] == -1:
            dist[r][c] = d
            cola.append((r, c))
```

El código en Python: el recorrido

```
def bosque(tablero, tripletas):
    # Menor número de pasos de (0, 0) a (R-1, C-1); -1 si no se puede.
    prohibida = marcar(tablero, tripletas)
    R = len(tablero)
    C = len(tablero[0])
    dist = []
    for r in range(R):
        dist.append([-1] * C)
    cola = deque()
    entrar(prohibida, dist, cola, 0, 0, 0)
    while len(cola) > 0:
        (r, c) = cola.popleft()
        d = dist[r][c] + 1
        entrar(prohibida, dist, cola, r - 1, c, d)
        entrar(prohibida, dist, cola, r + 1, c, d)
        entrar(prohibida, dist, cola, r, c - 1, d)
        entrar(prohibida, dist, cola, r, c + 1, d)
    return dist[R - 1][C - 1]
```

El código en C++: marcar

```cpp
vector<vector<bool> > marcar(const vector<string> &tablero,
                             const vector<vector<int> > &tripletas) {
  // Matriz con las casillas por las que no se puede pasar.
  int R = tablero.size();
  int C = tablero[0].size();
  vector<vector<bool> > prohibida(R, vector<bool>(C, false));
  for (int r = 0; r < R; r++) {
    for (int c = 0; c < C; c++) {
      prohibida[r][c] = tablero[r][c] == '#';
    }
  }
  for (size_t i = 0; i < tripletas.size(); i++) {
    int ri = tripletas[i][0], ci = tripletas[i][1], radio = tripletas[i][2];
    for (int r = max(0, ri - radio); r < min(R, ri + radio + 1); r++) {
      for (int c = max(0, ci - radio); c < min(C, ci + radio + 1); c++) {
        if ((r - ri) * (r - ri) + (c - ci) * (c - ci) <= radio * radio) {
          prohibida[r][c] = true;
        }
      }
    }
  }
  return prohibida;
}
```

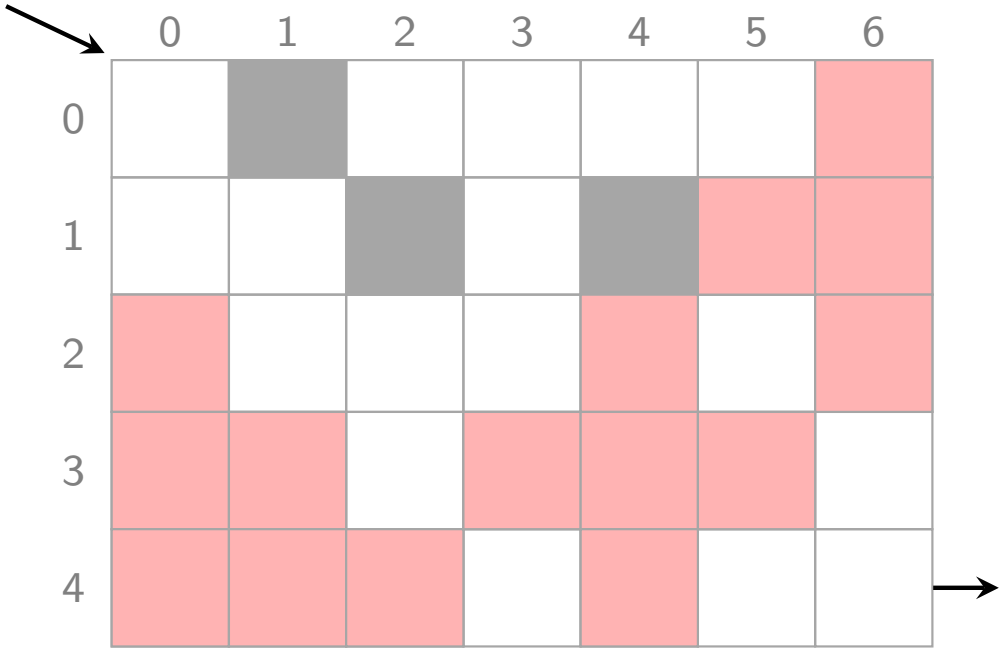
El código en C++: entrar a la cola

```cpp
void entrar(const vector<vector<bool> > &prohibida, vector<vector<int> > &dist,
            queue<pair<int, int> > &cola, int r, int c, int d) {
  // Mete (r, c) a la cola si existe, se puede pisar y es nueva.
  int R = dist.size();
  int C = dist[0].size();
  if (r >= 0 && r < R && c >= 0 && c < C) {
    if (!prohibida[r][c] && dist[r][c] == -1) {
      dist[r][c] = d;
      cola.push(make_pair(r, c));
    }
  }
}
```

El código en C++: el recorrido

```
int bosque(const vector<string> &tablero,
           const vector<vector<int> > &tripletas) {
  // Menor número de pasos de (0, 0) a (R-1, C-1); -1 si no se puede.
  vector<vector<bool> > prohibida = marcar(tablero, tripletas);
  int R = tablero.size();
  int C = tablero[0].size();
  vector<vector<int> > dist(R, vector<int>(C, -1));
  queue<pair<int, int> > cola;
  entrar(prohibida, dist, cola, 0, 0, 0);
  while (!cola.empty()) {
    int r = cola.front().first;
    int c = cola.front().second;
    cola.pop();
    int d = dist[r][c] + 1;
    entrar(prohibida, dist, cola, r - 1, c, d);
    entrar(prohibida, dist, cola, r + 1, c, d);
    entrar(prohibida, dist, cola, r, c - 1, d);
    entrar(prohibida, dist, cola, r, c + 1, d);
  }
  return dist[R - 1][C - 1];
}
```

Los bordes



No se puede

Si además de las dos anteriores está $(1, 6, 1)$, la salida queda sin camino: su distancia sigue en -1 y eso se devuelve. Si se marca la entrada, la cola arranca vacía y sale lo mismo.

Los bordes

Lo que el código ya atiende

- Una zona se sale del tablero: `max` y `min` recortan.
- Dos zonas se pisan: pintar dos veces deja lo mismo.
- `radio` $= 0$: solo la casilla de la criatura.
- Tablero de 1×1 sin marcar: entrada y salida coinciden, 0 pasos.

Lo que cuesta

De qué depende

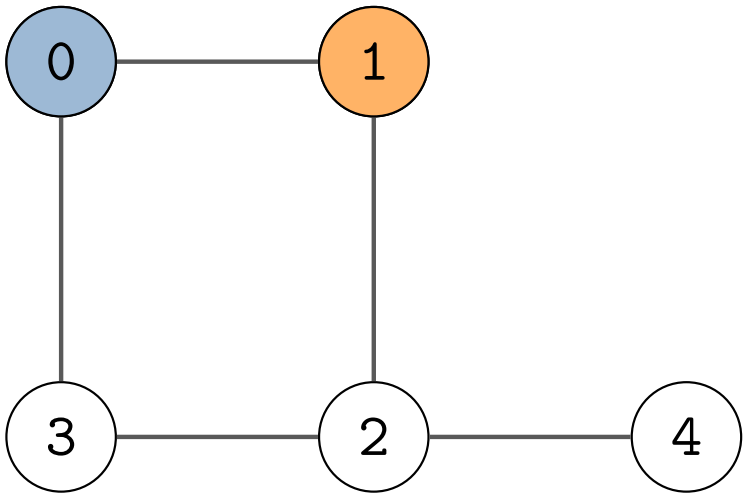
Marcar corre N veces el doble ciclo sobre el cuadrado de lado $2 \cdot \texttt{radio} + 1$, recortado al tablero. Sin recorte da $N(2 \cdot \texttt{radio} + 1)^2$; con recorte, cada tripleta toca a lo sumo $R \cdot C$ casillas. El recorrido mete cada casilla a la cola una vez y mira cuatro vecinas:

$$O(N \cdot R \cdot C) \text{ para marcar,} \qquad \Theta(R \cdot C) \text{ para recorrer.}$$

Contenido

3 Dos colores: recorrido en profundidad modificado
- El modelo
- La traza
- El código
- Los bordes
- Lo que cuesta

El modelo: nodos, aristas y color



Qué es cada cosa

Cada arista obliga a que sus extremos tengan colores distintos. El arreglo `color` hace de marca de visitado: -1 sin pintar, 0 azul, 1 naranja. Aquí `color` es `[0, 1, -1, -1, -1]`.

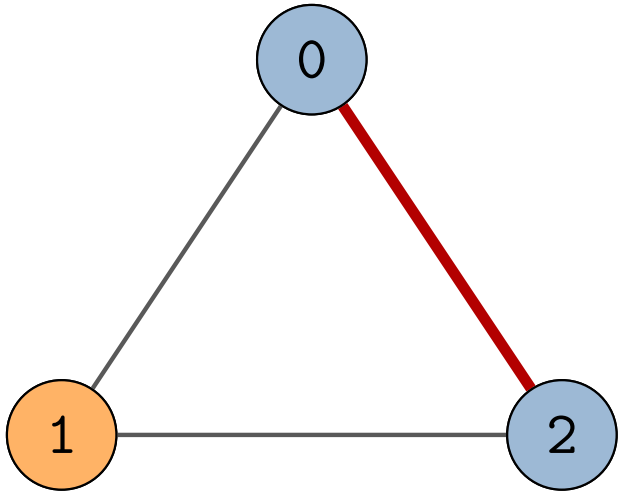
El modelo: lo que cambia en el recorrido

Desde u se mira a v	Qué significa	Qué se hace
v sin pintar	primera visita	llamar con el otro color
v con otro color	arista cumplida	nada
v con el mismo color	arista imposible	anotar el choque

El otro color

Si u tiene color c, el otro es $1 - c$: de 0 sale 1, y de 1 sale 0.

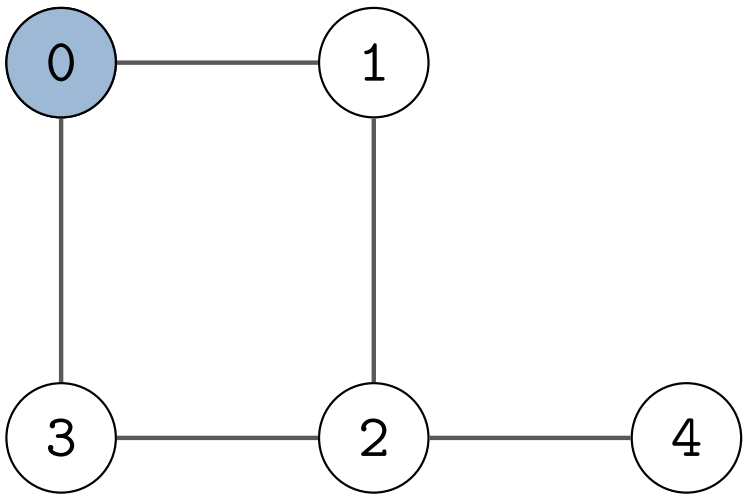
El modelo: lo que devuelve cada llamada



Cierto o falso

Un choque es una arista con los dos extremos del mismo color. Cada llamada a `pintar` devuelve cierto si no hubo choque y falso si lo hubo, en ella o en las que abrió. Quien recibe falso también devuelve falso: el falso sube.

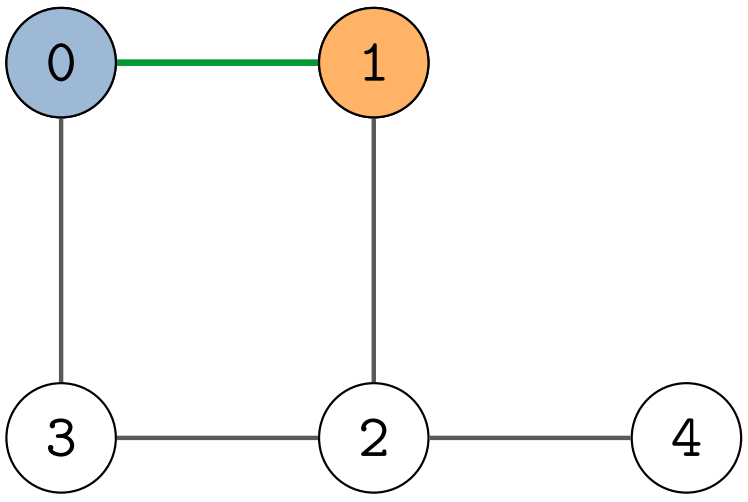
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`

Se arranca: el nodo 0 queda azul.

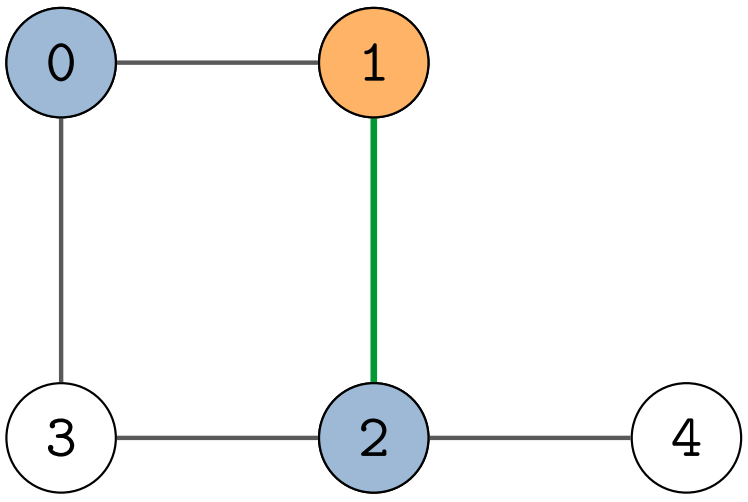
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`

Desde 0 se llega a 1, sin color: queda naranja.

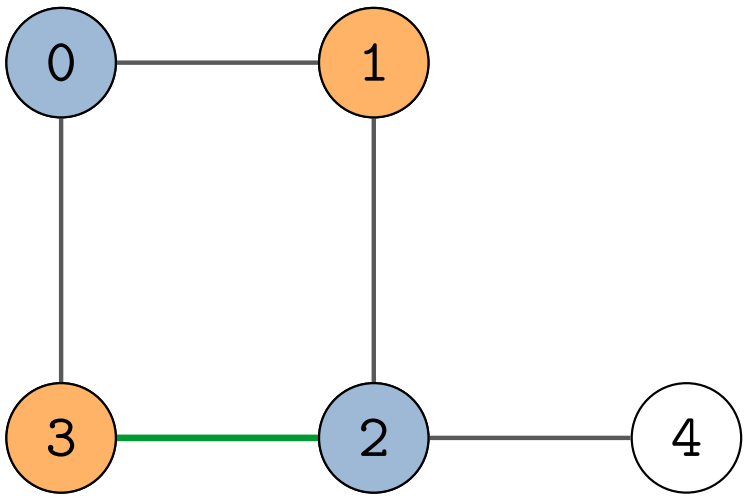
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`

Desde 1 se llega a 2, sin color: queda azul.

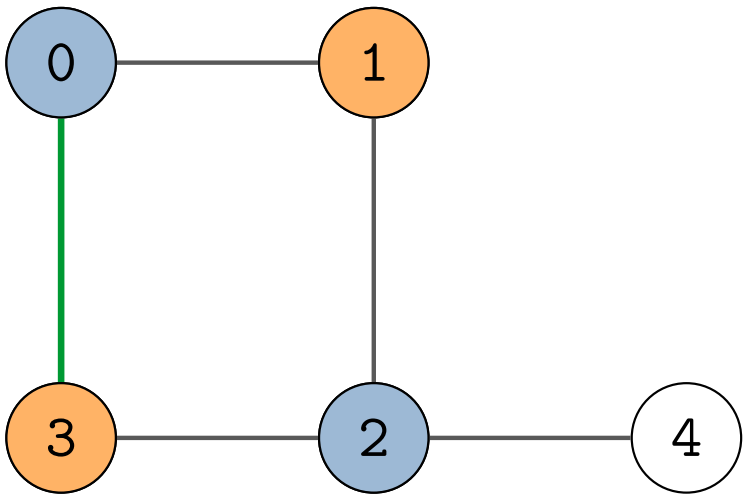
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`
`pintar(3, naranja)`

Desde 2 se llega a 3, sin color: queda naranja.

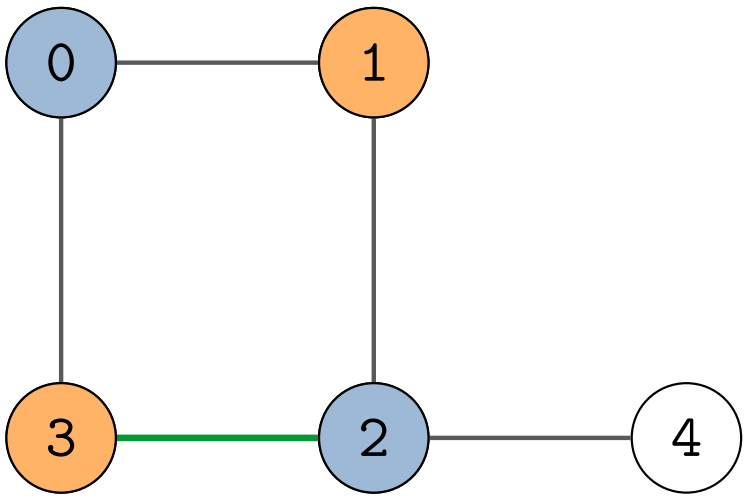
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`
`pintar(3, naranja)`

El nodo 3, naranja, mira a 0, que ya es azul: la arista se cumple.

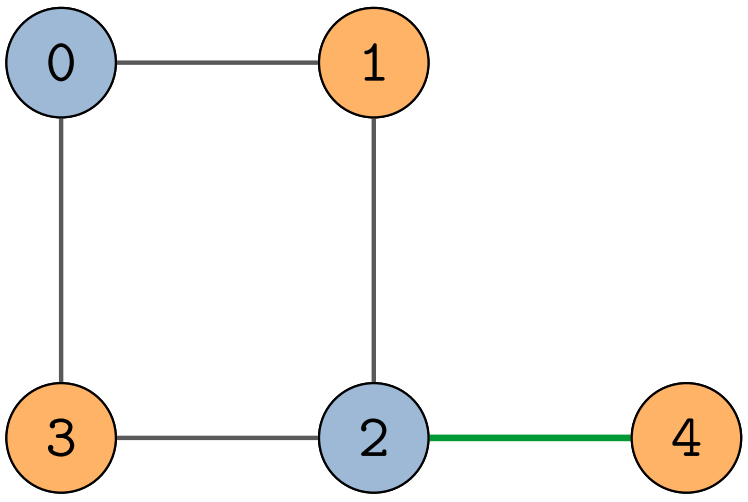
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`

El nodo 3 ya miró a sus vecinos, sin choque: devuelve cierto a `pintar(2, azul)`.

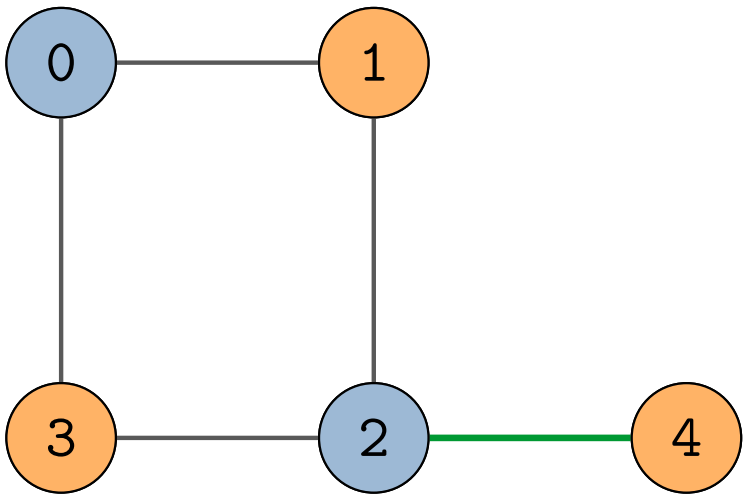
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`
`pintar(4, naranja)`

Desde 2 se llega a 4, sin color: queda naranja.

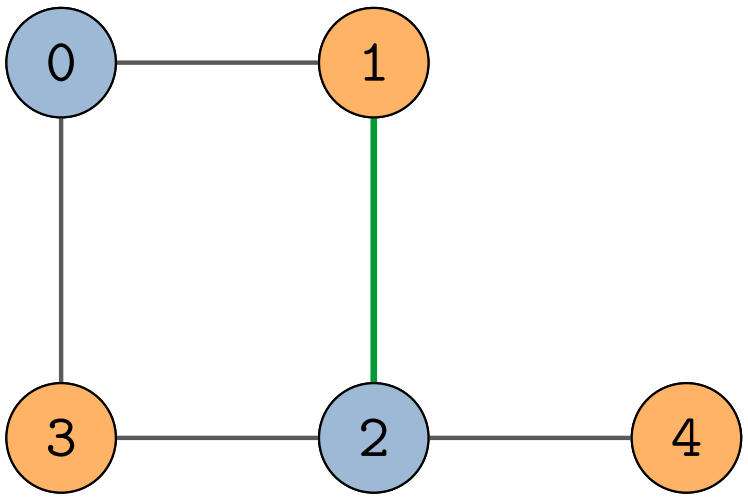
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`

El nodo 4 ya miró a sus vecinos, sin choque: devuelve cierto a `pintar(2, azul)`.

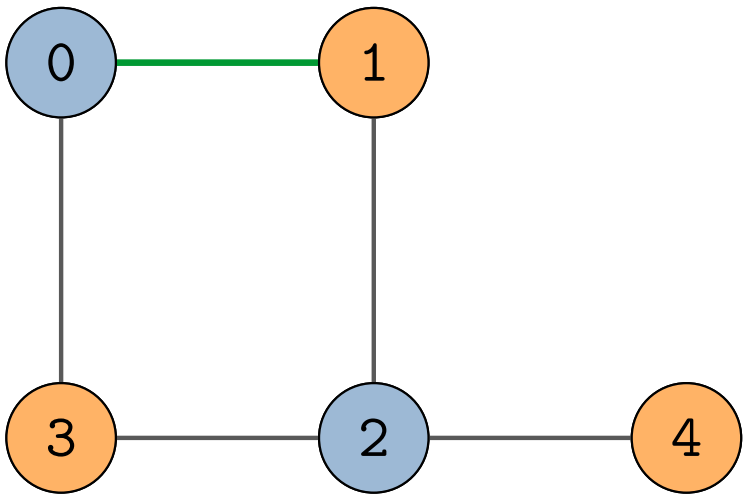
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`

El nodo 2 ya miró a sus vecinos, sin choque: devuelve cierto a `pintar(1, naranja)`.

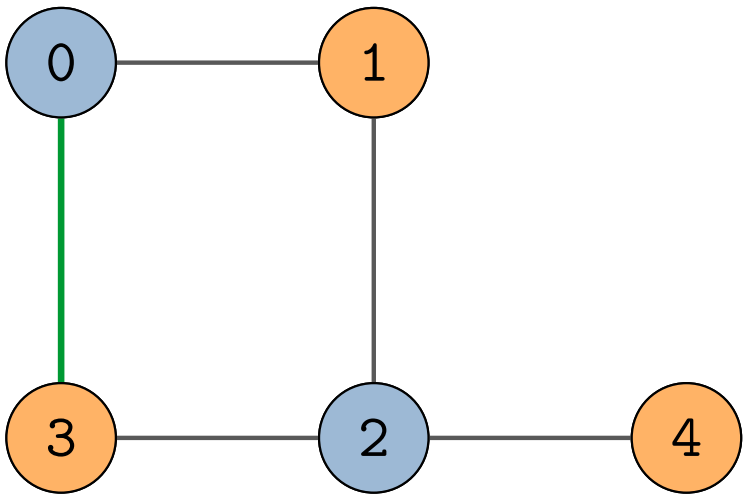
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`

El nodo 1 ya miró a sus vecinos, sin choque: devuelve cierto a `pintar(0, azul)`.

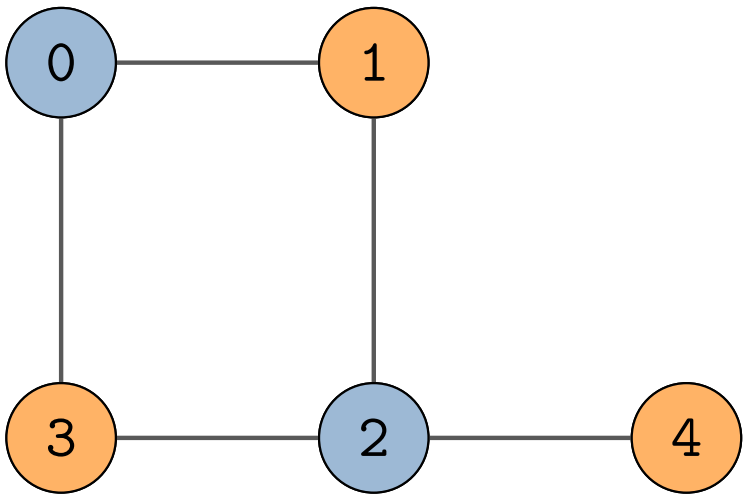
La traza: un ciclo par y una hoja



Llamadas abiertas
`pintar(0, azul)`

El nodo 0, azul, mira a 3, que ya es naranja: la arista se cumple.

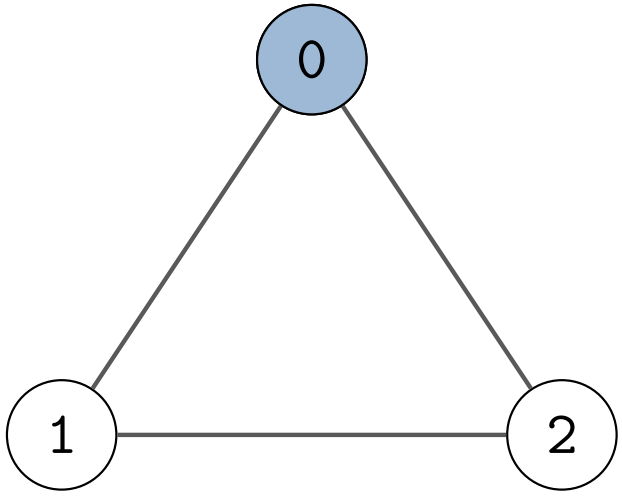
La traza: un ciclo par y una hoja



Llamadas abiertas
ninguna

El nodo 0 ya miró a sus vecinos, sin choque: `pintar(0, azul)` devuelve cierto y el grafo es bipartito.

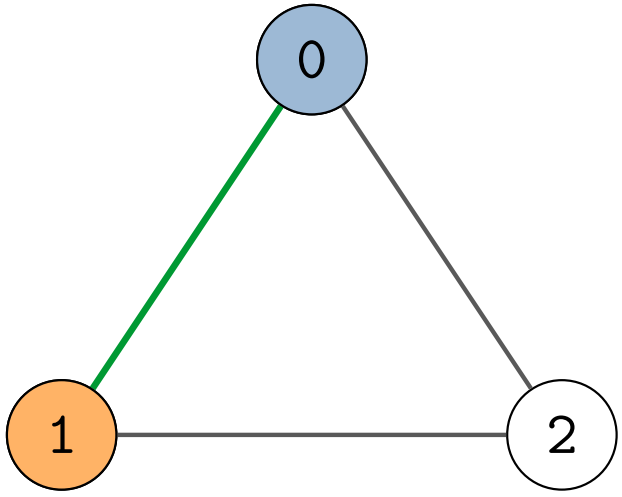
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`

Se arranca: el nodo 0 queda azul.

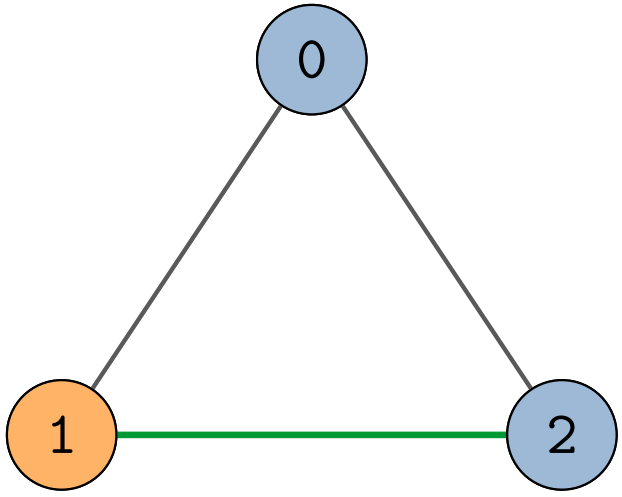
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`

Desde 0 se llega a 1, sin color: queda naranja.

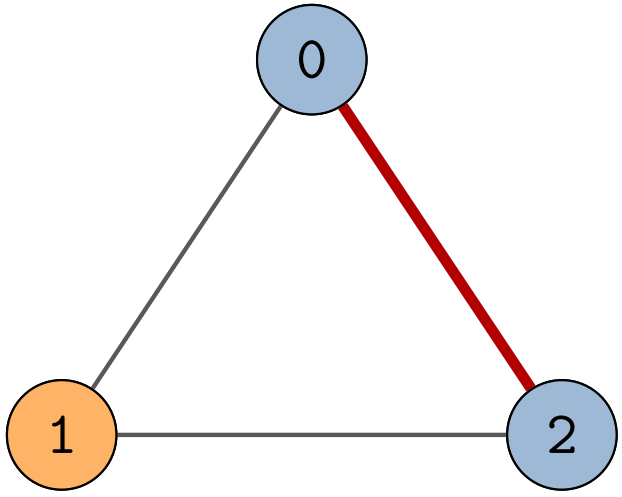
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`

Desde 1 se llega a 2, sin color: queda azul.

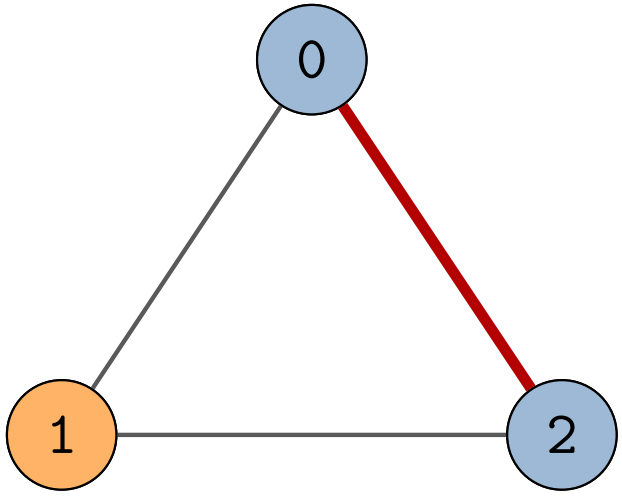
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`
`pintar(2, azul)`

El nodo 2 mira a 0 y los dos tienen color azul: choque.

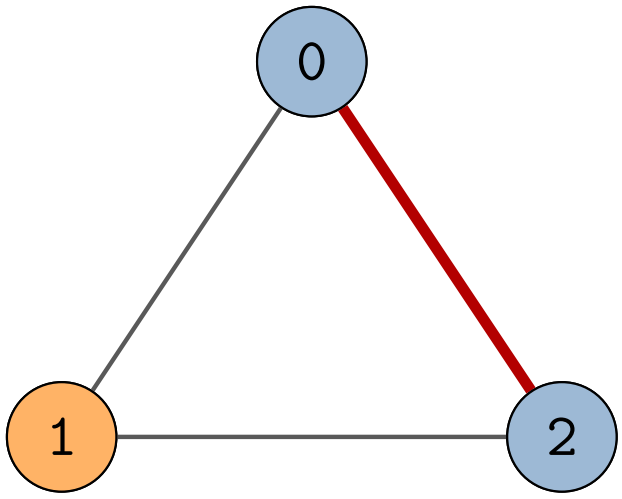
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`
`pintar(1, naranja)`

El nodo 2 ya miró a sus vecinos y hubo choque: devuelve falso a `pintar(1, naranja)`.

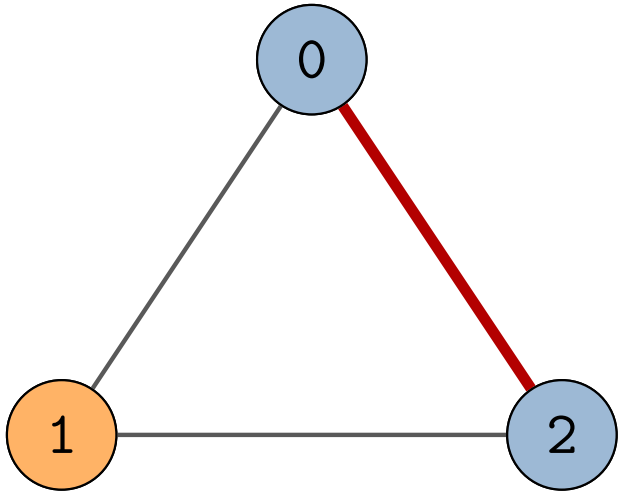
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`

`pintar(1, naranja)` recibió falso: también devuelve falso a `pintar(0, azul)`.

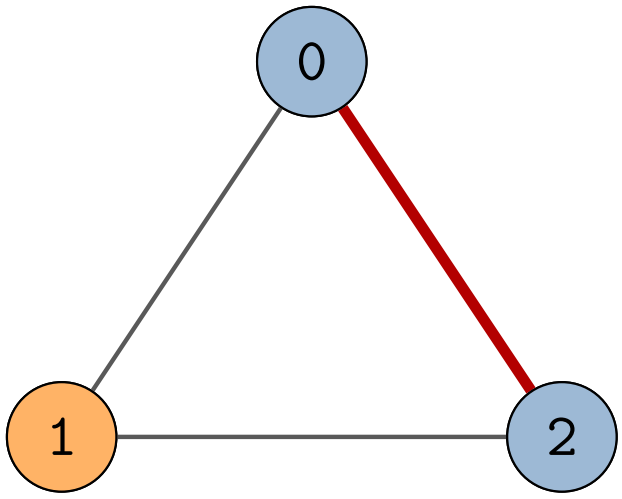
La traza: un triángulo



Llamadas abiertas
`pintar(0, azul)`

El nodo 0 mira a 2 y los dos tienen color azul: choque.

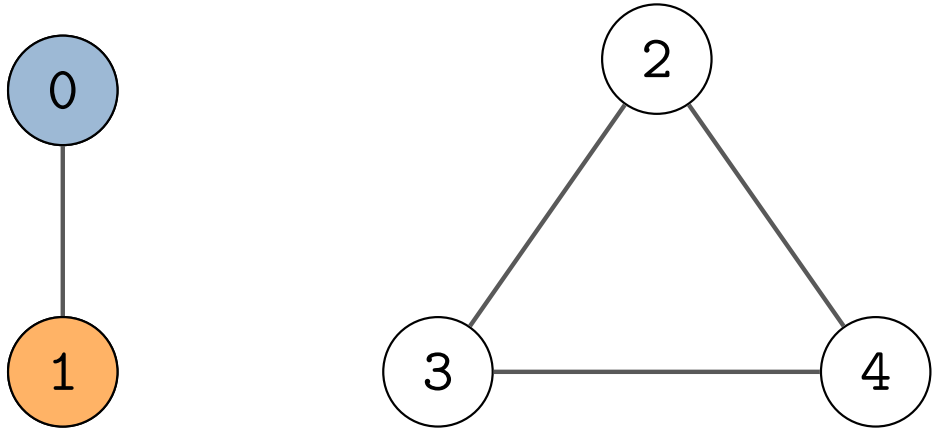
La traza: un triángulo



Llamadas abiertas
ninguna

`pintar(0, azul)` devuelve falso: el falso subió por las llamadas y el grafo no es bipartito.

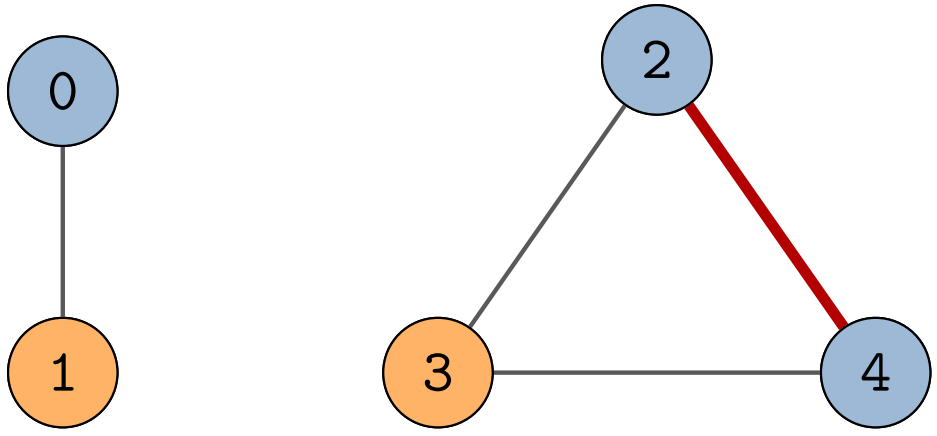
¿Un solo arranque basta?



Pregunta

Arrancando en 0 se pintan 0 y 1, sin choque. ¿Ya se puede decir que el grafo es bipartito?

Un arranque por pedazo



Respuesta: no

El grafo no es conexo: el recorrido no salta de un pedazo a otro, y el triángulo quedó sin ver. Por eso hay un ciclo afuera: cada nodo sin color arranca un recorrido nuevo.

El código en Python: pintar

```
def pintar(adj, color, u, c):
    # Pinta u con c y sigue; devuelve si no hubo choque.
    color[u] = c
    sin_choque = True
    for v in adj[u]:
        if color[v] == -1:
            if not pintar(adj, color, v, 1 - c):
                sin_choque = False
        elif color[v] == c:
            sin_choque = False
    return sin_choque
```

El código en Python: un arranque por pedazo

```
def bipartito(adj):
    # Un recorrido por cada pedazo del grafo.
    color = [-1] * len(adj)
    resultado = True
    for u in range(len(adj)):
        if color[u] == -1:
            if not pintar(adj, color, u, 0):
                resultado = False
    return resultado
```

El código en C++: pintar

```cpp
bool pintar(const vector<vector<int> > &adj, vector<int> &color,
            int u, int c) {
  // Pinta u con c y sigue; devuelve si no hubo choque.
  color[u] = c;
  bool sin_choque = true;
  for (size_t i = 0; i < adj[u].size(); i++) {
    int v = adj[u][i];
    if (color[v] == -1) {
      if (!pintar(adj, color, v, 1 - c)) {
        sin_choque = false;
      }
    } else if (color[v] == c) {
      sin_choque = false;
    }
  }
  return sin_choque;
}
```

El código en C++: un arranque por pedazo

```cpp
bool bipartito(const vector<vector<int> > &adj) {
  // Un recorrido por cada pedazo del grafo.
  vector<int> color(adj.size(), -1);
  bool resultado = true;
  for (size_t u = 0; u < adj.size(); u++) {
    if (color[u] == -1) {
      if (!pintar(adj, color, u, 0)) {
        resultado = false;
      }
    }
  }
  return resultado;
}
```

Los bordes

La profundidad de las llamadas

En un camino largo cada nodo abre una llamada. Medido con los valores por omisión: Python da `RecursionError` pasados 998 nodos, y este C++, con una pila de 8192 KB, se revienta hacia los 104 700. Para grafos así se cambia la recursión por una pila propia, o por un recorrido en amplitud que alterne colores.

Los bordes: lo que el código ya atiende

Caso	Sale
Sin aristas o sin nodos	cierto
Nodo con arista a sí mismo	falso
Aristas repetidas	como si fuera una

Lo que cuesta

De qué depende

Cada nodo se pinta una sola vez, y al pintarlo se recorre su lista. Una arista aparece en dos listas. El ciclo de afuera mira V nodos:

$$\Theta(V + E).$$

En memoria van el arreglo de colores y la pila de llamadas, de hasta V posiciones cada uno.

Lo que tienen en común

Se pregunta	Textos	Bosque	Colores
¿Qué entra y qué sale?	páginas y k; un número	tablero y tripletas; pasos	grafo; cierto o falso
¿Cuánto cuesta lo directo?	$\binom{M-1}{K-1}$	al menos $\binom{R+C-2}{R-1}$	2^n
¿Qué lo simplifica?	fijar el tope	marcar antes el peligro	el color es forzado
¿Qué se recorre?	candidatos t	casillas libres	nodos por pedazo
¿Qué borde lo rompe?	`lo` $= 0$	entrada prohibida	varios pedazos

Referencias

- T. Cormen, C. Leiserson, R. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, 2009. Capítulo 22: recorridos en amplitud y en profundidad.
- S. Halim, F. Halim y S. Effendy. *Competitive Programming 4*: bisección sobre la respuesta.